24MCDAAT302

Design & Analysis of Algorithms

Module - 3

Backtracking

Backtracking name itself suggests that we are going back and coming forward; if it satisfies the condition, then return success, else we go back again. It is used to solve a problem in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criteria.

When to use a Backtracking algorithm?

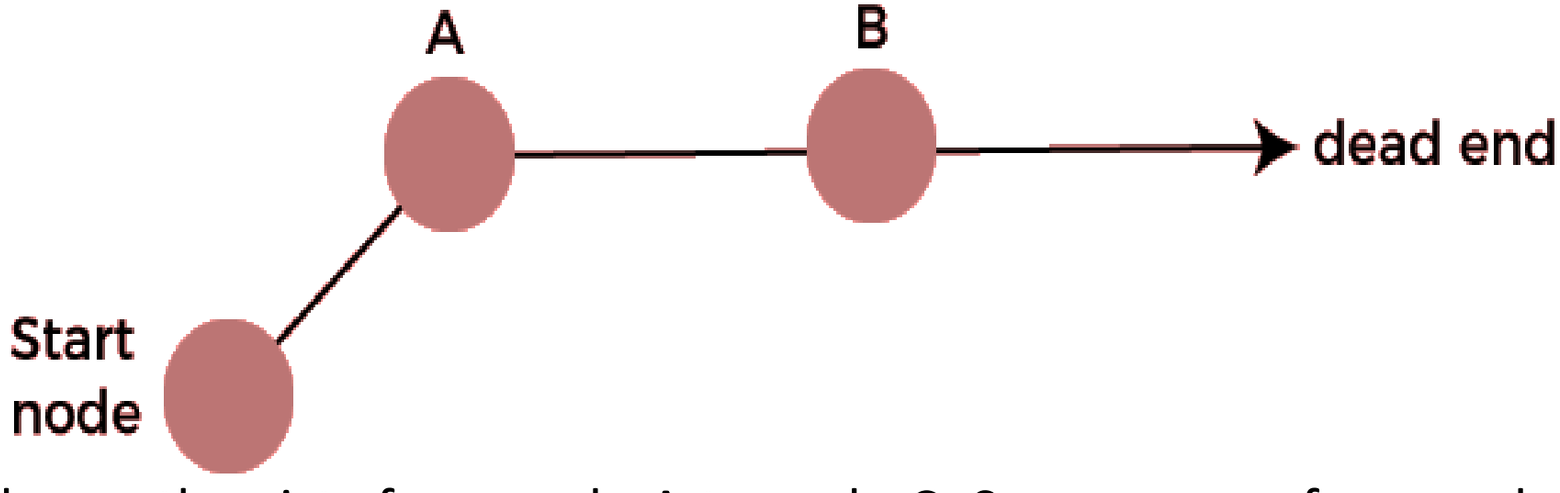
When we have multiple choices, then we make the decisions from the available

choices. In the following cases, we need to use the backtracking algorithm:

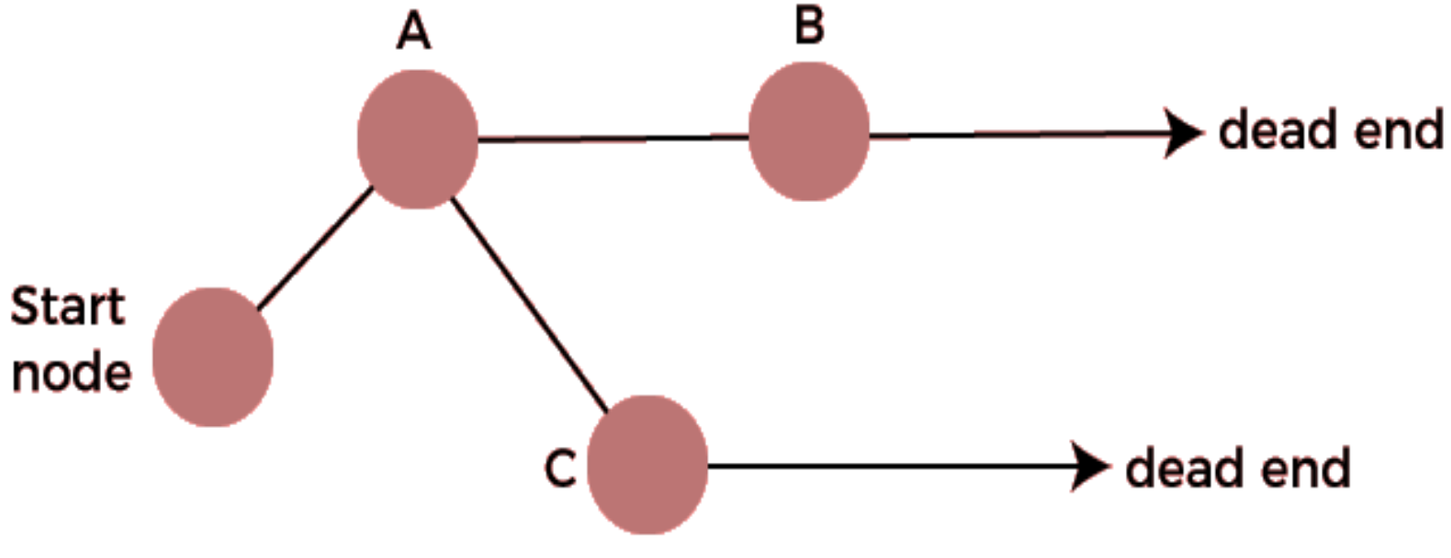
- A piece of sufficient information is not available to make the best choice, so we use the backtracking strategy to try out all the possible solutions.
- Each decision leads to a new set of choices. Then again, we backtrack to make new decisions. In this case, we need to use the backtracking strategy.

Backtracking working

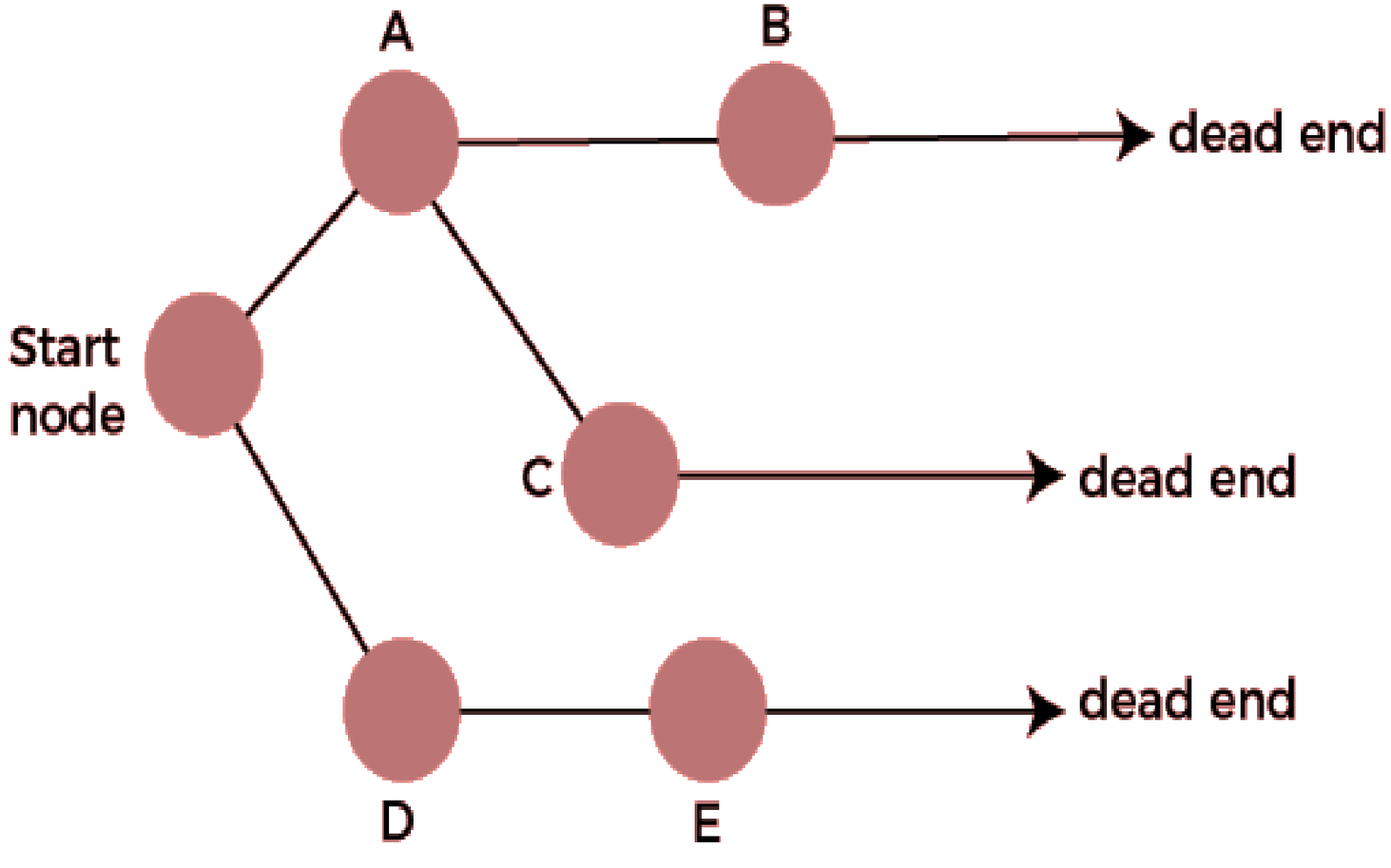
We start with a start node. First, we move to node A. Since it is not a feasible solution so we move to the next node, i.e., B. B is also not a feasible solution, and it is a dead-end so we backtrack from node B to node A.



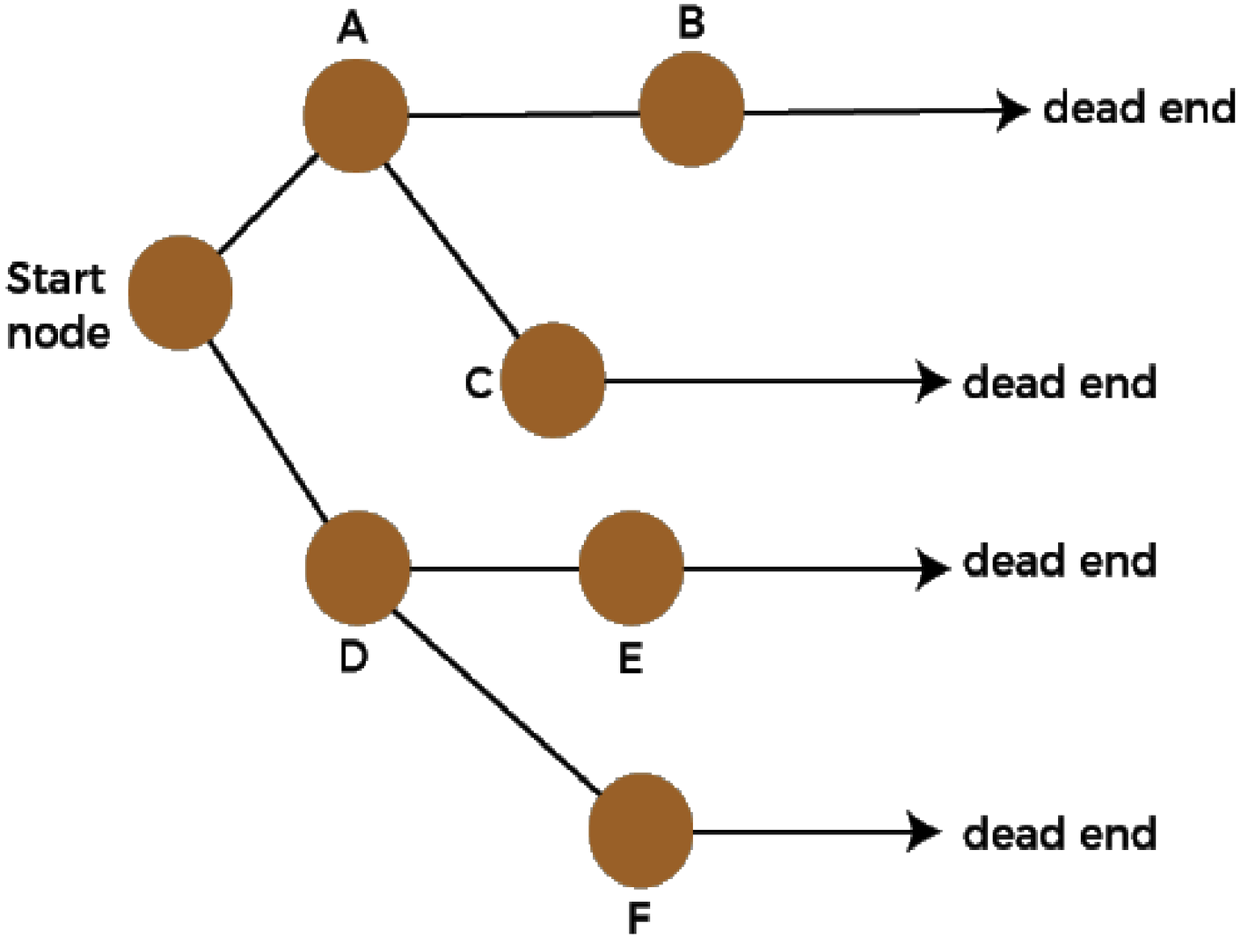
Suppose another path exists from node A to node C. So, we move from node A to node C. It is also a dead-end, so again backtrack from node C to node A. We move from node A to the starting node.



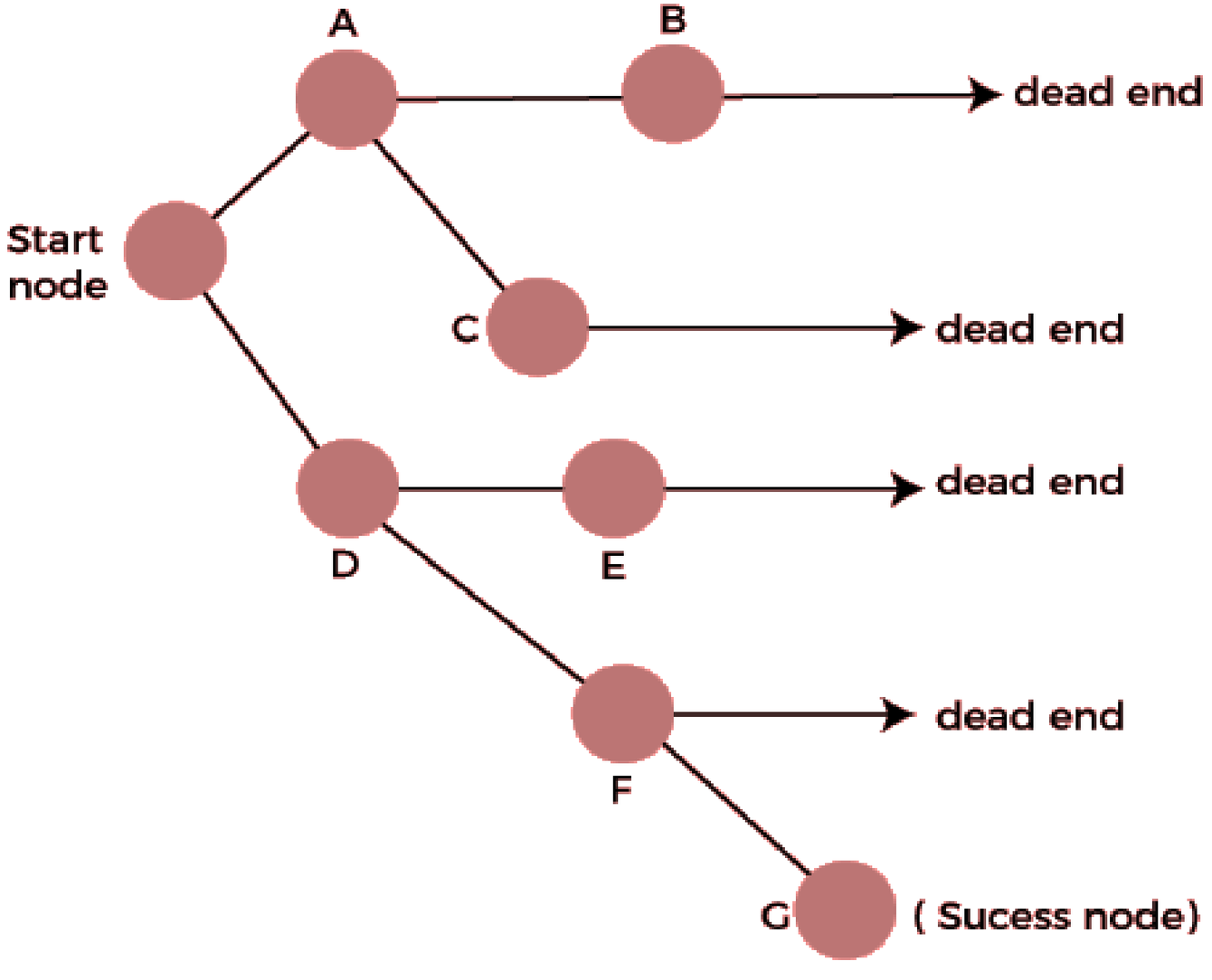
Now we will check any other path exists from the starting node. So, we move from start node to the node D. Since it is not a feasible solution so we move from node D to node E. The node E is also not a feasible solution. It is a dead end so we backtrack from node E to node D.



Suppose another path exists from node D to node F. So, we move from node D to node F. Since it is not a feasible solution and it's a dead-end, we check for another path from node F.



Suppose there is another path exists from the node F to node G so move from node F to node G. The node G is a success node.



The terms related to the backtracking are:

- **Live node:** The nodes that can be further generated are known as live nodes.
- **E node:** The nodes whose children are being generated and become a success node.
- **Success node:** The node is said to be a success node if it provides a feasible solution.
- **Dead node:** The node which cannot be further generated and also does not provide a feasible solution is known as a dead node.

Many problems can be solved by backtracking strategy, and that problems satisfy complex set of constraints, and these constraints are of two types:

- **Implicit constraint:** It is a rule in which how each element in a tuple is related.
- **Explicit constraint:** The rules that restrict each element to be chosen from the given set.

Backtracking Principle: Theoretical View

Backtracking is a problem-solving strategy used to find solutions by exploring all possible options and eliminating those that fail early.

It is based on a trial-and-error approach:

- Choose a possible option,
- Move forward,
- If the current path leads to a solution, keep going,
- If it doesn't, go back (backtrack) and try another path.

General Characteristics

- Used for constraint satisfaction problems
- Explores the solution recursively
- Employs systematic search with pruning
- Based on the idea of building a solution incrementally

Backtracking Algorithm

Step 1: Define the Problem

Represent the problem in a way where:

- The solution can be built piece by piece.
- At each step, multiple choices are possible.
- A rule exists to check if the current solution is valid.

Step 2: Choose the First Option

Start from the initial state.

Pick the **first decision** (choice) among the possible candidates.

Step 3: Apply the Choice

Add this choice to the partial solution.

Proceed to the next level or decision point.

Step 4: Check for Validity

Before moving forward, check if the current partial solution is **valid**:

- Does it meet the required constraints?
- Is it still possible to reach a full solution from here?

Step 5: Explore Further

If valid:

- Continue to the next decision level (**recurse**).

If not valid:

- **Backtrack**: Undo the last choice and try the next available option.

Step 6: Repeat the Process

Repeat the process of choosing, applying, checking, and backtracking recursively until:

- A valid solution is found, OR
- All options are exhausted.

Step 7: Terminate

Once a valid complete solution is found:

- Accept it (either return or print).

If no solutions are possible:

- Return that the problem has **no solution**.

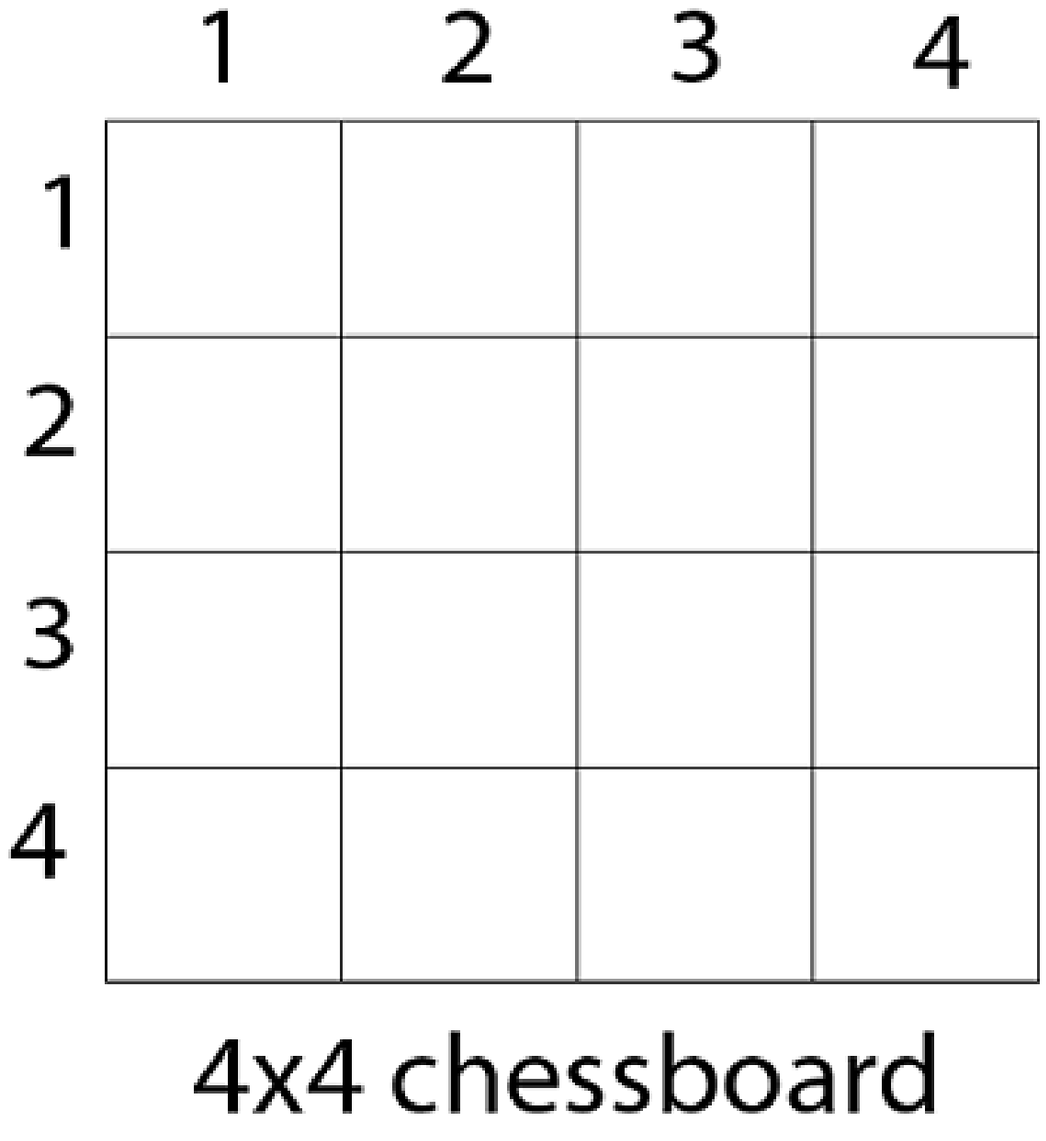
N-Queens Problem

N-Queens Problem

N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal.

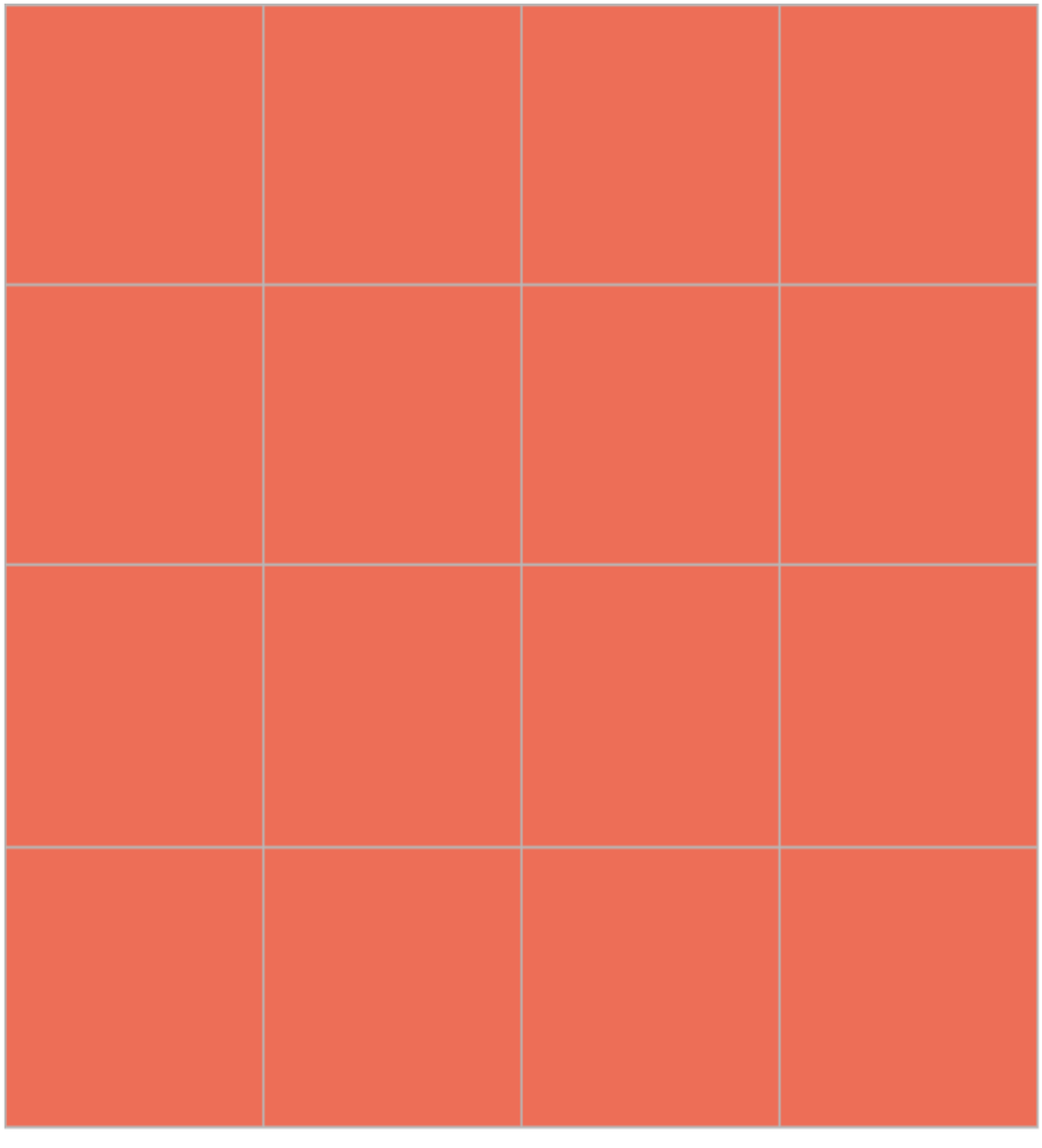
It can be seen that for n =1, the problem has a trivial solution, and no solution exists for n =2 and n =3. So first we will consider the 4 queens problem and then generate it to n - queens problem.

Given a 4 x 4 chessboard and number the rows and column of the chessboard 1 through 4.



4x4 chessboard

Since, we have to place 4 queens such as q1 q2 q3 and q4 on the chessboard, such that no two queens attack each other.



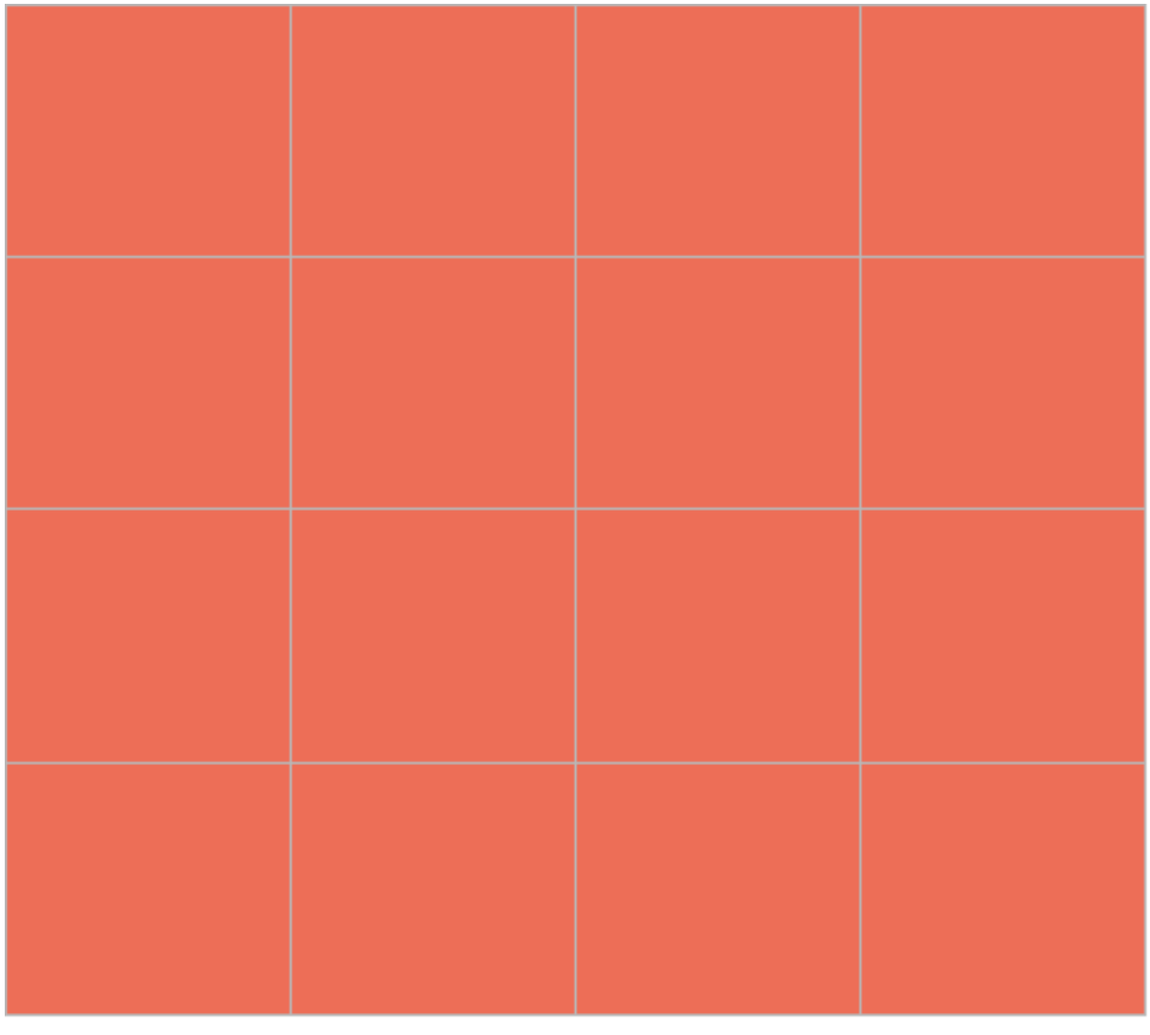
4x4 Chessboard

Queens to be placed



Using Backtracking to Solve N Queens

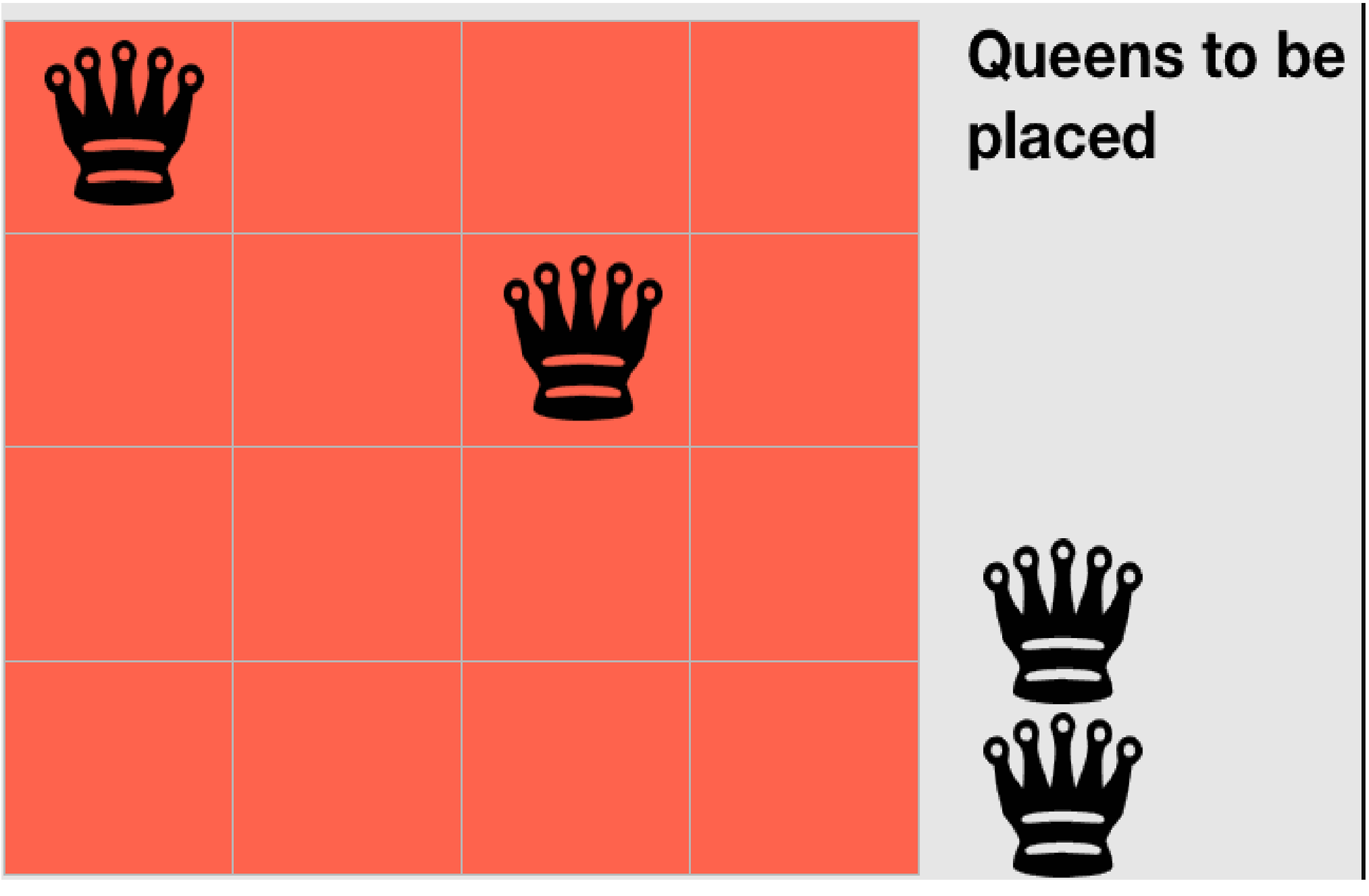
The above picture shows a 4x4 chessboard and we have to place 4 queens on it. So, we will start by placing the first queen in the first row.



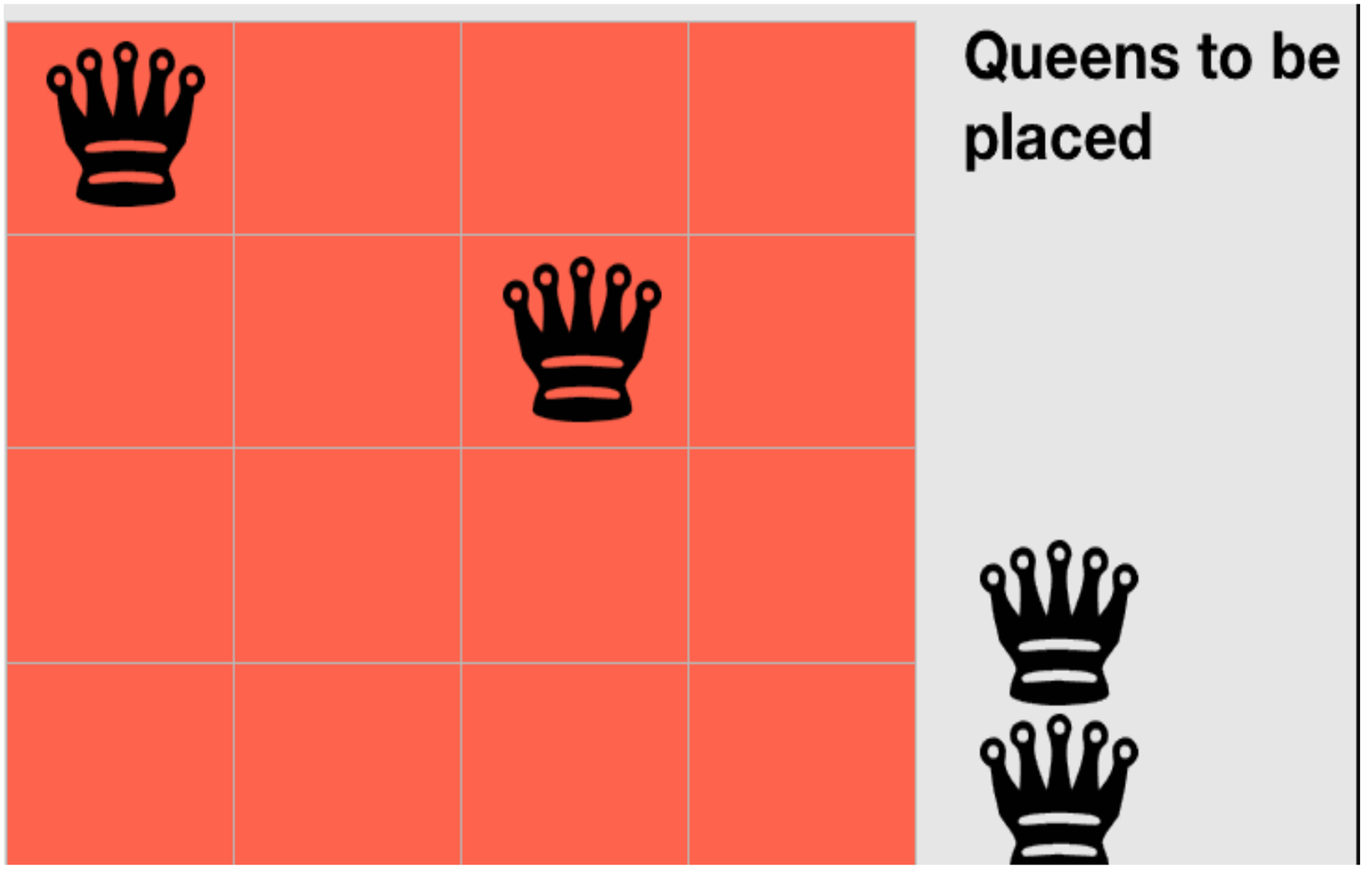
4x4 Chessboard



Now, the second step is to place the second queen in a safe position. Also, we can't place the queen in the first row, so we will try putting the queen in the second row this time.

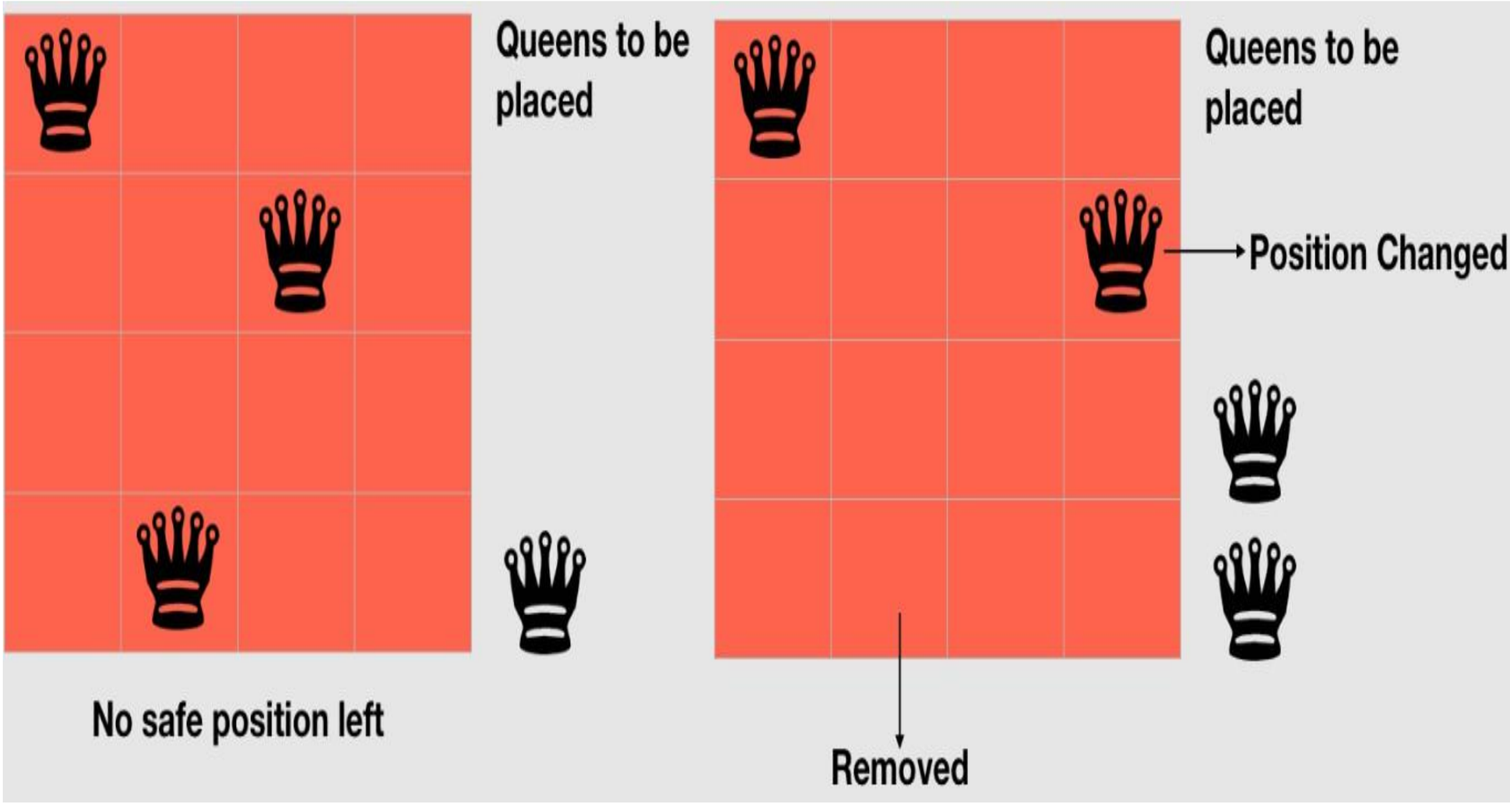


Let's place the third queen in a safe position, somewhere in the third row.

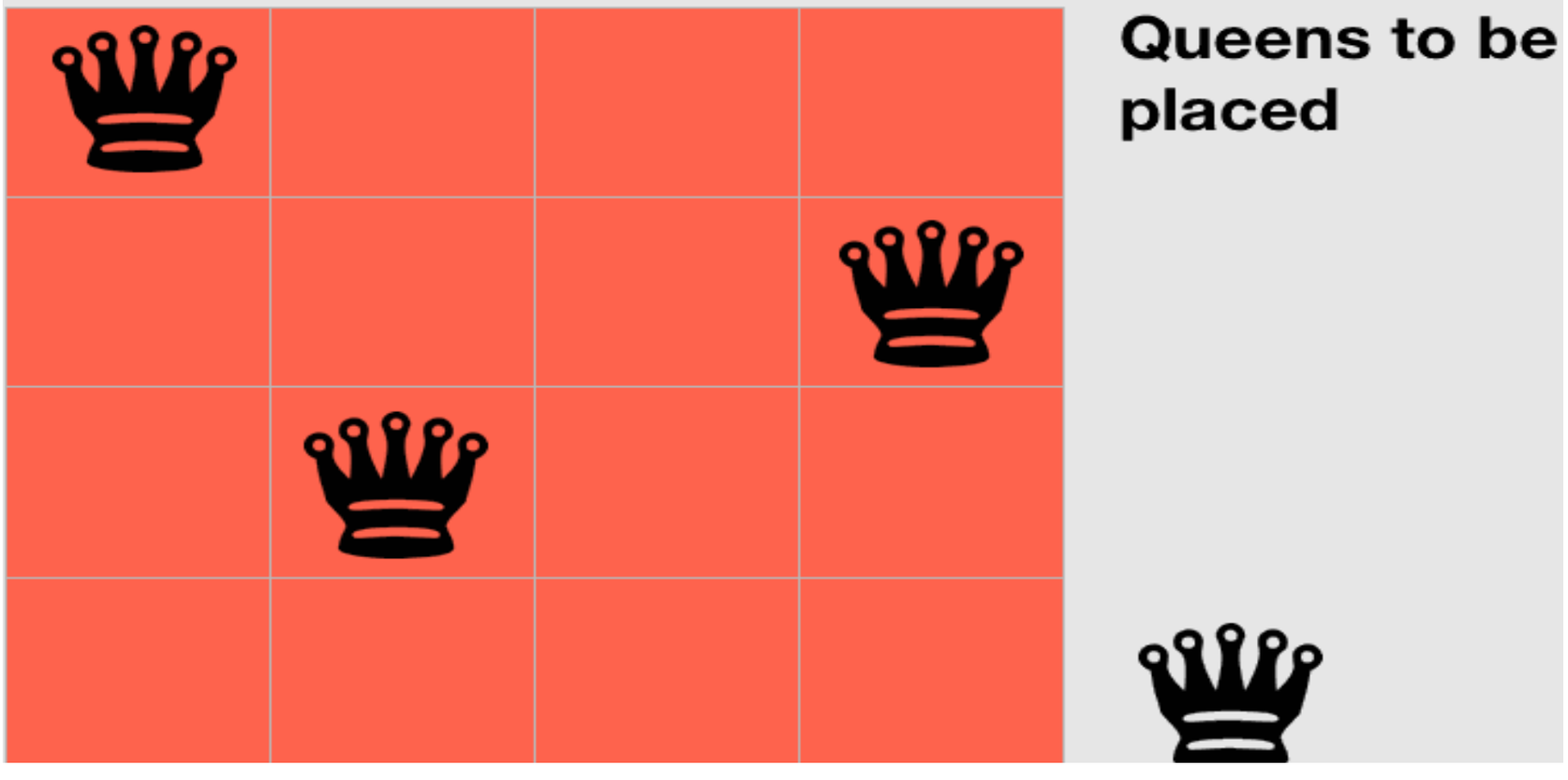


Now, we can see that there is no safe place where we can put the last queen. So, we will just change the position of the previous queen i.e., backtrack and change the previous decision.

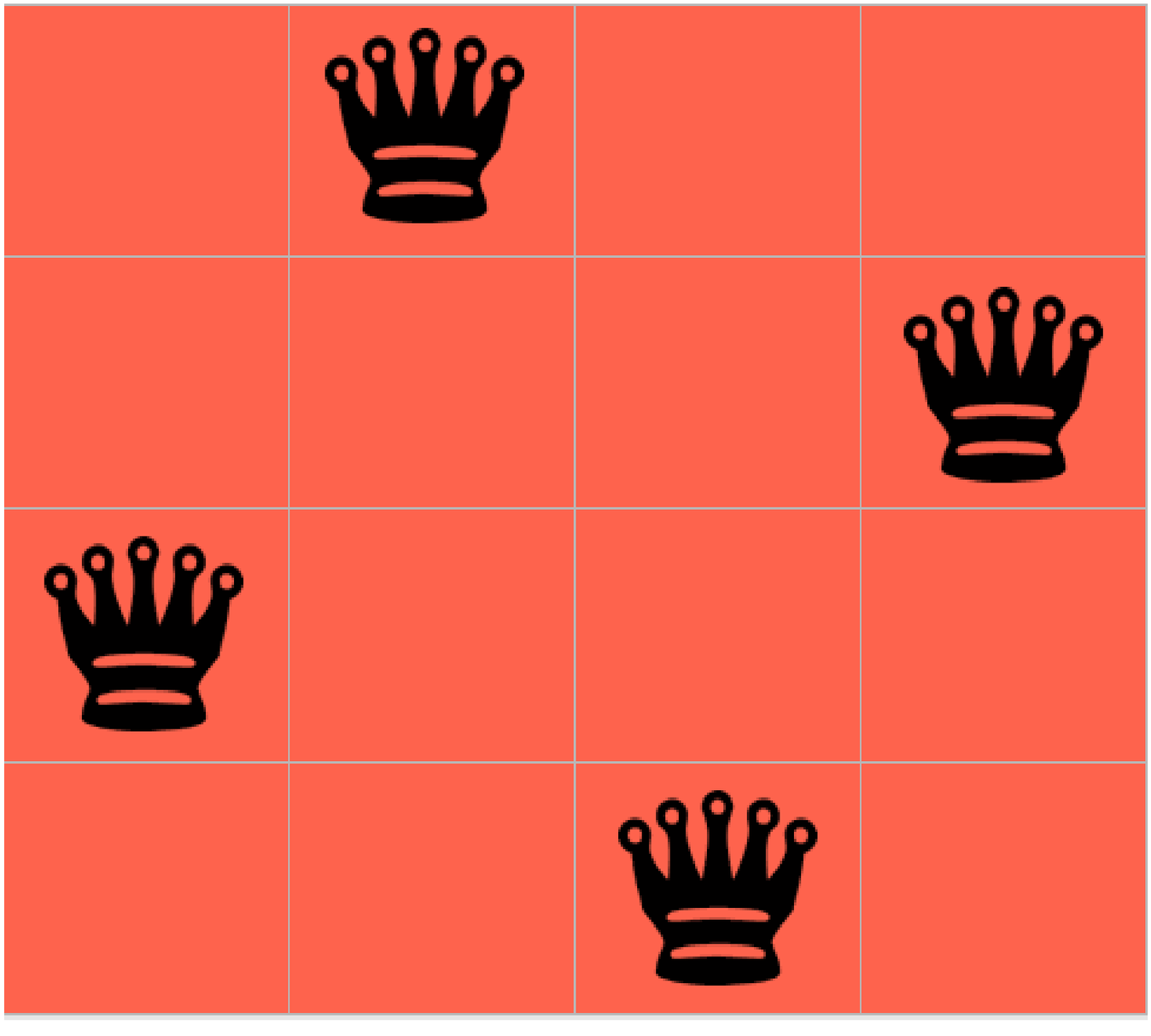
Also, there is no other position where we can place the third queen, so we will go back one more step and change the position of the second queen.



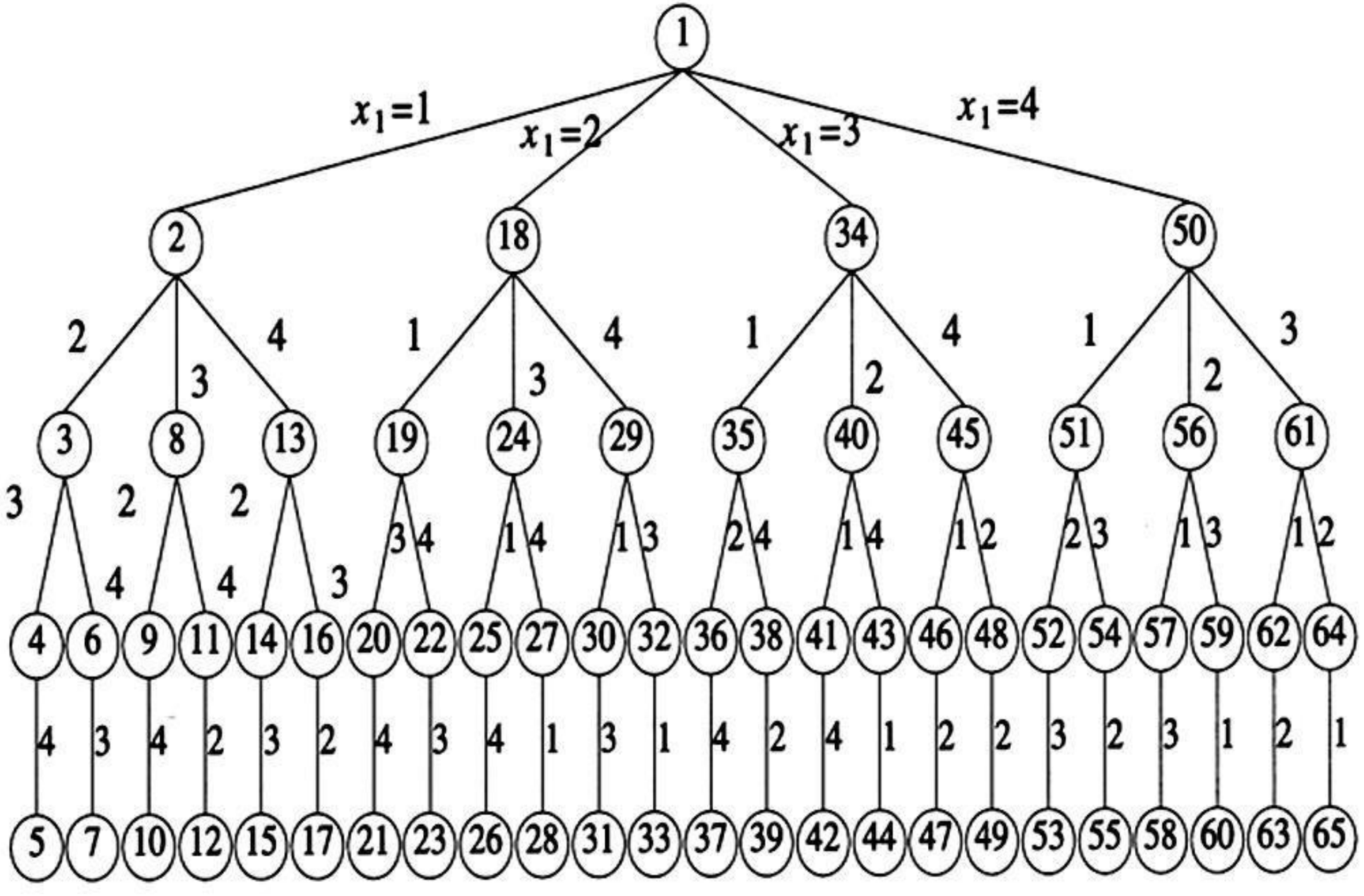
And now we will place the third queen again in a safe position other than the previously placed position in the third row.



We will continue this process and finally, we will get the solution as shown below.



State Space Tree of the Four-queens Problem



4 - Queen's problem

- In 4- queens problem, we have 4 queens to be placed on a 4*4 chessboard, satisfying the constraint that no two queens should be in the same row, same column, or in same diagonal.
- The solution space according to the external constraints consists of 4 to the power 4, 4-tuples i.e., Si = {1, 2, 3, 4} and 1<= I <=4, whereas according to the internal constraints they consist of 4! solutions i.e., permutation of 4.

Solution of 4 – queen's with the help of backtracking

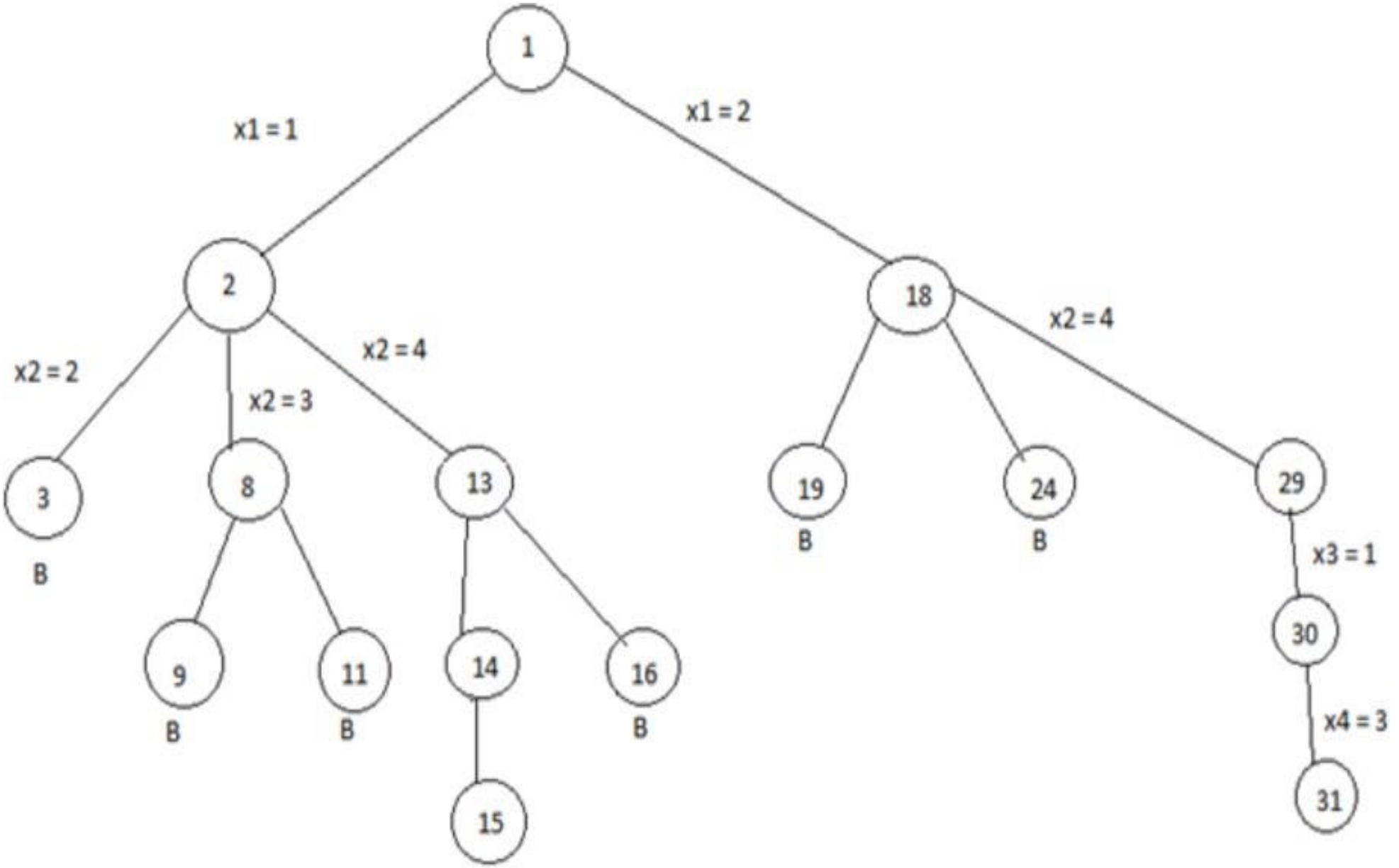
We can solve 4-queens problem through backtracking by taking it as a bounding function in use the criterion that if (x1, x2, ……., xi) is a path to a current E-node, then all the children nodes with parent-child labelings x (i+1) are such that (x1, x2, x3, ….., x(i+1)) represents a chessboard configuration in which no queens are attacking.

So we start with the root node as the only live node. This time this node becomes the E-node and the path is (). We generate the next child. Suppose we are generating the child in ascending order. Thus the node number 2 is generated and path is now 1 i.e., the queen 1 is placed in the first row and in the first column.

Now, node 2 becomes the next E-node or line node. Further, try the next node in the ascending nodes i.e., the node 3 which is having x2 = 2 means queen 2 is placed in the second column but by this the queen 1 and 2 are on the same diagonal, so node 3 becomes dead here so we backtrack it and try the next node which is possible.

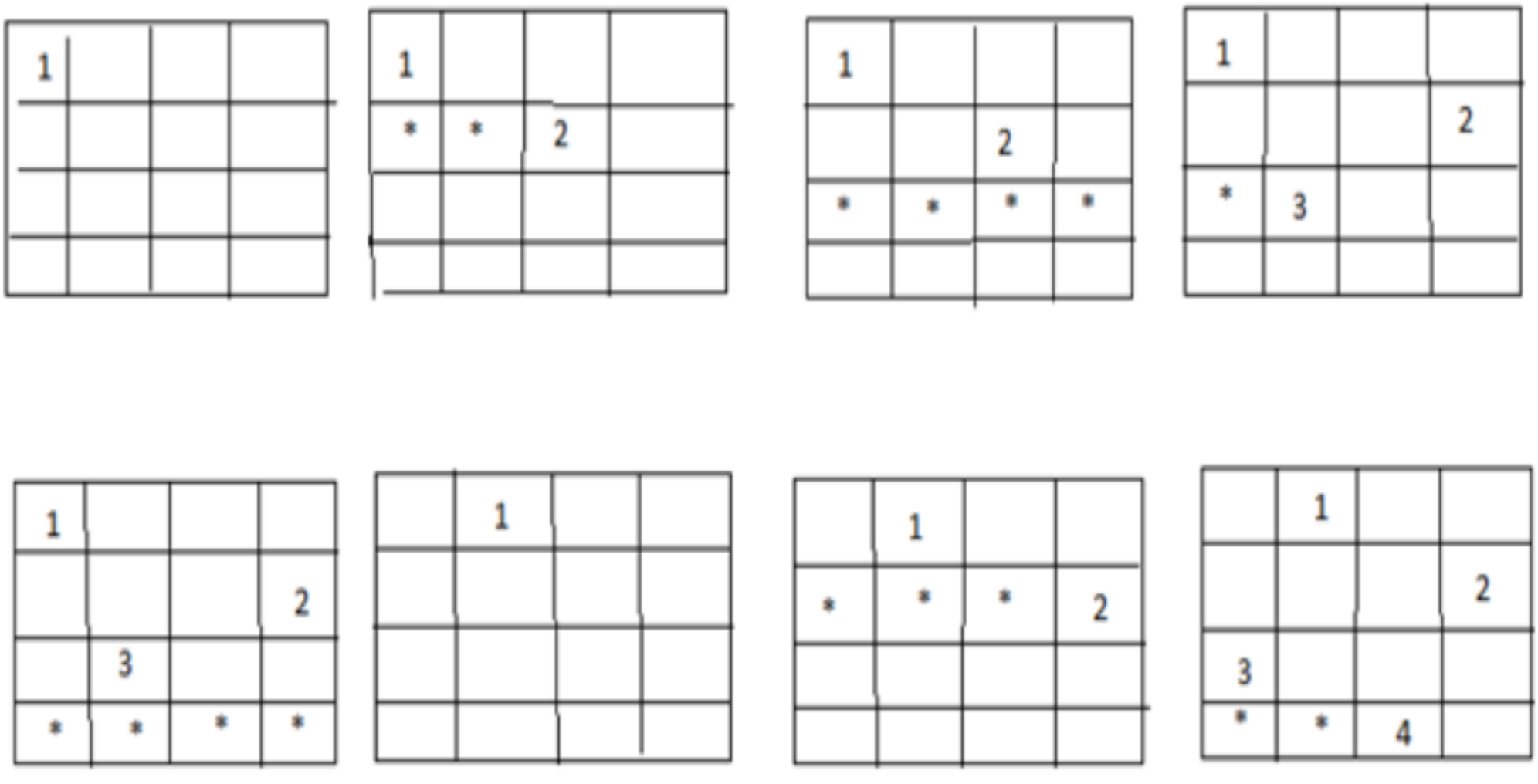
Here, the x2 = 3 means the queen 2 is placed in the 3rd column. As it satisfies all the constraints so it becomes the next live node.

After this try for next node 9 having x3 = 2 which means the queen 3 placed in the 2nd column, but by this the 2 and 3 queen are on the same diagonal so it becomes dead. Now we try for next node 11 with x3 = 4, but again the queens 2 and 3 are on the same diagonal so it is also a dead node.

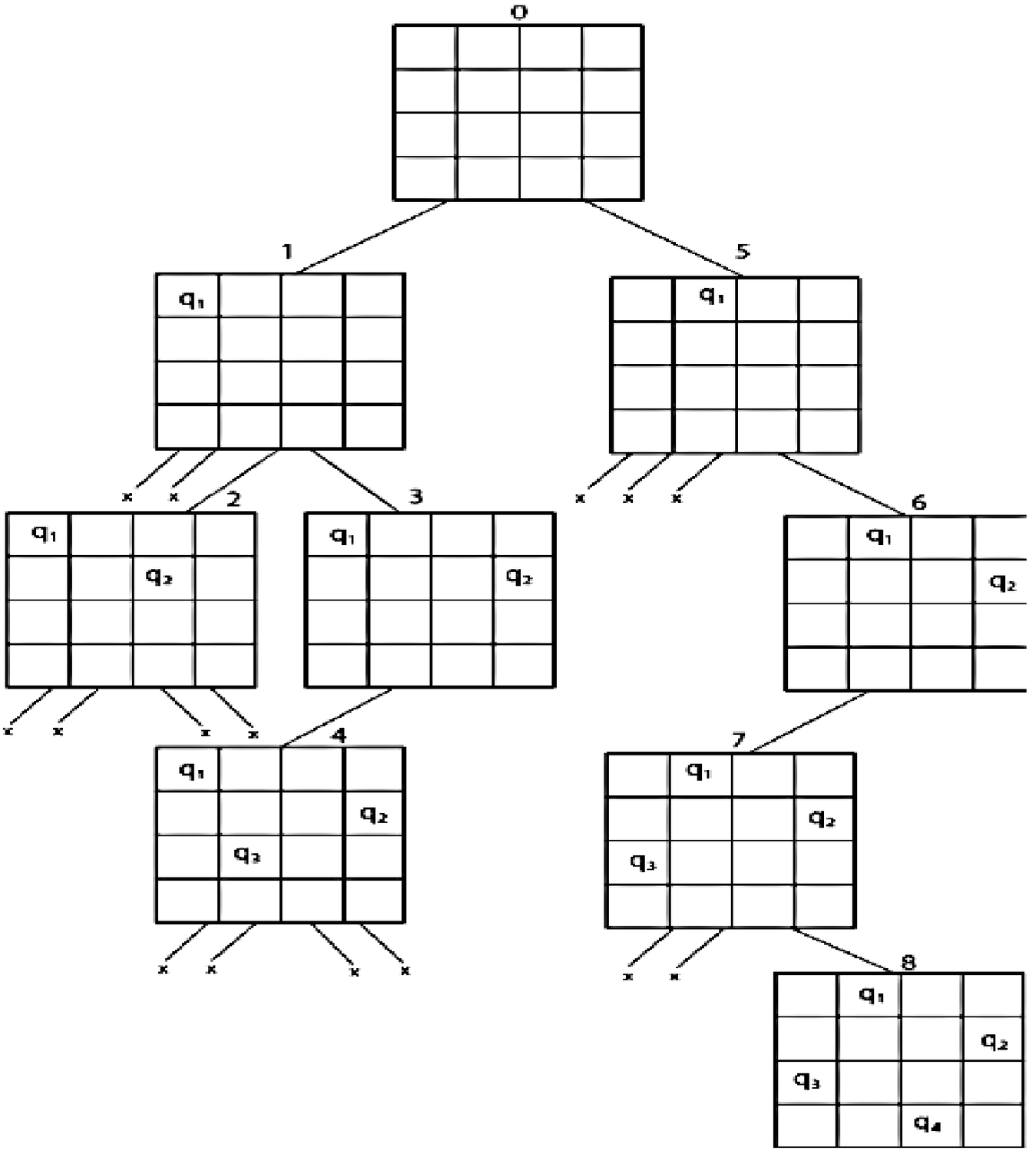

1
x1 = 1
x1 = 2
2
18
x2 = 2
x2 = 3
x2 = 4
x2 = 4
3
8
13
19
24
29
B
B
B
x3 = 1
30
9
11
14
16
x4 = 3
B
B
B
31
15

* The B denotes the dead node.

We try for all the possible positions for the queen 3 and if not any position satisfy all the constraints then backtrack to the previous live node.



The implicit tree for 4 - queen problem for a solution (2, 4, 1, 3) is as follows:



Now, the node13 become the new live node with x2 = 4, means queen 2 is placed in the 4th column. Move to the next node 14. It becomes the next live node with x3 = 2 means the queen 3 is placed in the 2nd column. Further, we move to the next node 15 with x4 = 3 as the live node. But this makes the queen 3 and 4 on the same diagonal resulting this node 15 is the dead node so we have to backtrack to the node 14 and then backtrack to the node 13 and try the other possible node 16 with x3 = 3 by this also we get the queens 2 and 3 on the same diagonal so the node is the dead node.

So we further backtrack to the node 2 but no other node is left to try so the node 2 is killed so we backtrack to the node 1 and try another sub-tree having x1 = 2 which means queen 1 is placed in the 2nd column.

Now again with the similar reason, nodes 19 and 24 are killed and so we try for the node 29 with x2 = 4 means the queen 2 is placed in the 4th column then we try for the node 30 with x3 = 1 as a live node and finally we proceed to next node 31 with x4 = 3 means the queen 4 is placed in 3rd column

Here, all the constraints are satisfied, so the desired result for 4 queens is {2, 4, 1, 3}.

N - Queen's problem

The n – queen problem is the generalized problem of 8-queens or 4 – queen's problem. Here, the n – queens are placed on a n * n chess board, which means that the chessboard has n rows and n columns and the n queens are placed on thus n * n chessboard such that no two queens are placed in the same row or in the same column or in same diagonal. So that, no two queens attack each other.

Here, we suppose that the queen i is to be placed in row i. We can say that 1 queen is placed in the first row only but can have columns from 1, 2... n so that they satisfy all the explicit and implicit constraints.

All solutions to the n – queen's problem can be therefore represented as n – tuples (x1, x2... xn) where xi is the column on which queen i is to be placed.

The explicit constraints using this formulation are Si = {1, 2, 3... n-1, n}, where 1 <= I <= n. Therefore, the solution space consists of nˆn n- tuples. Now, while considering the implicit constraints, that no two xi's can be same i.e., two queens cannot be in the same row, same column, or in same diagonal. So, each xi should be different. Now by the above constraint, the solution is the permutations of the n – tuples (1, 2, 3, 4... n-1, n).

Backtracking Algorithm

The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes then we backtrack and return false.

1)Start in the leftmost column
2)If all queens are placed return true
3)Try all rows in the current column.
Do following for every tried row.
a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.
b)If placing the queen in [row, column] leads to a solution then return true.
c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.
3) If all rows have been tried and nothing worked, return false to trigger backtracking.

Algorithm for N Queen problem

Step 1: Start

Step 2: Given n queens, read n from user and let us denote the queen number by
k. k=1,2,..,n.

Step 3: We start a loop for checking if the kth queen can be placed in the respective column of the kth row.

Step 4: For checking that whether the queen can be placed or not, we check if the
previous queens are not in diagonal or in same row with it.

Step 5: If the queen cannot be placed backtracking is done to the previous queens until a feasible solution is not found.

Step 6: Repeat the steps 3-5 until all the queens are placed.

Step 7: The column numbers of the queens are stored in an array and printed as a n-tuple solution

Step 8: Stop

8 - Puzzle Problem

⇒ In 8- Puzzle Problem, there are 8 tiles which are numbered from 1 to 8 placed on a 9- tile capacity square frame.

⇒ Objective of 8 - Puzzle Problem is to transform the arrangement of initial arrangement to a goal arrangement.

⇒

1	2	3
4		6
7	5	8

Initial arrangement

⟹

1	2	3
4	5	6
7	8	

Goal arrangement

⇒ There is always an empty slot in the initial arrangement.

⇒ Legal moves are the moves in which the tiles adjacent to the empty slot are moved to either Left, Right, Up, Down.

⇒ Initial arrangement is called as Initial state
Goal arrangement is called as goal state.

⟹ The State space Tree for 8-Puzzle is large because, there can be 9! different arrangements

⟹ In the state space Tree, the nodes are numbered as per the level

⟹ Each next move is generated based on empty slot Positions

⟹ Root node is called as node 1

⟹ Edges are labelled according to the direction in which empty slot moves

⟹ The root node becomes the E-node.

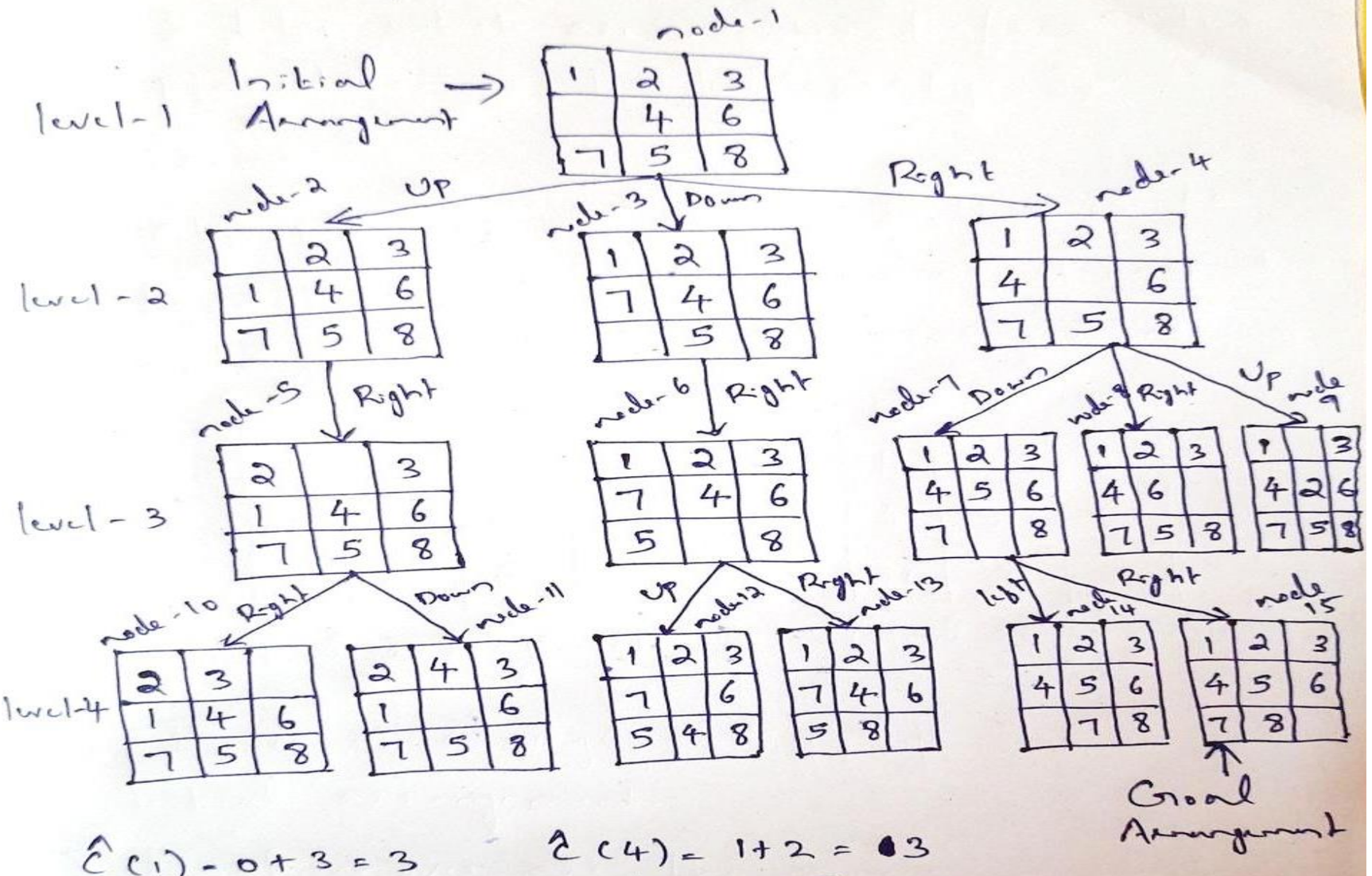
⟹ We can decide which node to become an E-node based on estimation formula.

$$\hat{c}(x) = f(x) + \hat{g}(x)$$

where $\hat{c}(x)$ = lower Bound Cost of node 'x'

$f(x)$ = length of the Path from root node to node 'x'

$\hat{g}(x)$ = Number of non-blank tiles which are not in their goal position.



$\hat{c}(1) = 0+3 = 3$

$\hat{c}(2) = 1+4 = 5$

$\hat{c}(3) = 1+4 = 5$

$\hat{c}(4) = 1+2 = 3$

$\hat{c}(5) = 2+5 = 7$

$\hat{c}(6) = 2+4 = 6$

$\hat{c}(7) = 2+1 = 3$

$\hat{c}(8) = 2+3 = 5$

$\hat{c}(9) = 2+3 = 5$

$\hat{c}(10) = 3+6 = 9$

$\hat{c}(11) = 3+5 = 8$

$\hat{c}(12) = 3+4 = 7$

$\hat{c}(13) = 3+4 = 7$

$\hat{c}(14) = 3+2 = 5$

$\hat{c}(15) = 3+0 = 3$

Sum of Subset Problem

Sum of Subset Problem

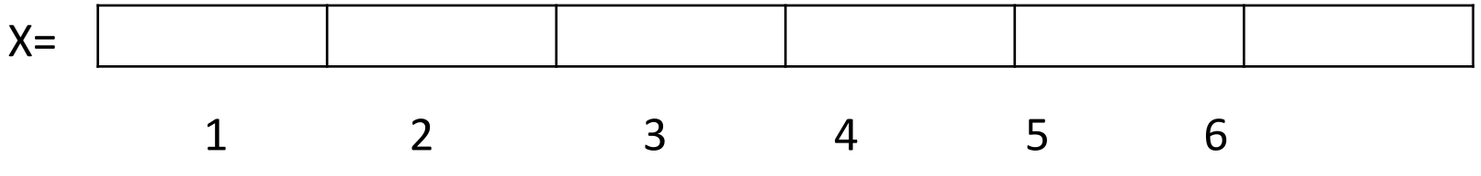
W[1:6] = {5,10,12,13,15,18}

Total 6 weights are there

n= 6

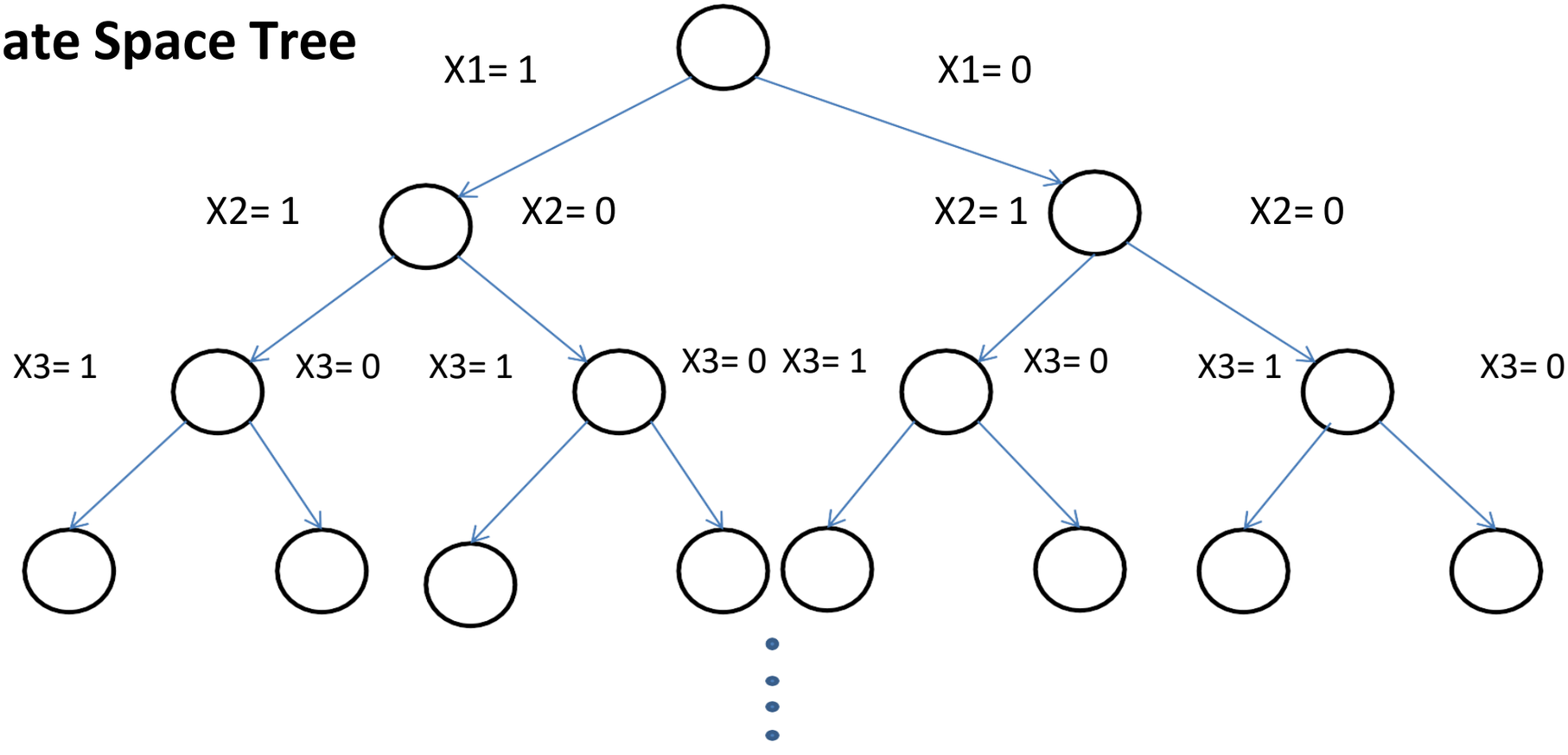
We have to take subset of weight such that their sum/ total is exactly equal to 'm'

m= 30



Each Xi values can be '0' or '1' whether am including it or not including

State Space Tree



Note : X1= 1 means First weight is included X1=0 means its not included

Like the above if I continue there are 6 weights there will be 7 levels including

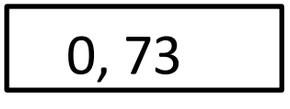
the root (Height will be 6). So total how many paths = 2^6 = 2^n.

So this is time consuming, its exponential. Amount of time taken to solve will be 2^n

- When we apply backtracking we try to kill the nodes if they are not satisfying bounding function

- We have to define the bounding function and we follow the bounding function
- Now we can see the solution using backtracking and also we can define the bounding function

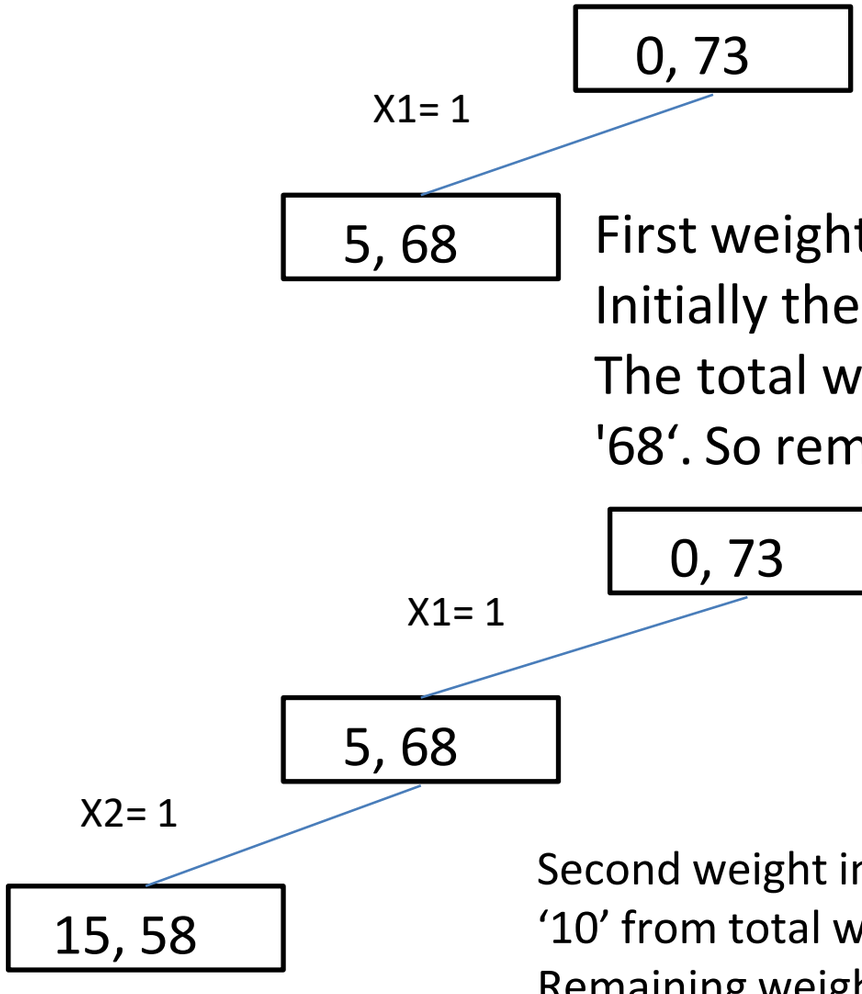
Lets generate the tree by putting the first node



In every node we will be putting 2 values
In first case it will '0' and the total of all weights ie, '73‘

W[1:6] = {5,10,12,13,15,18}

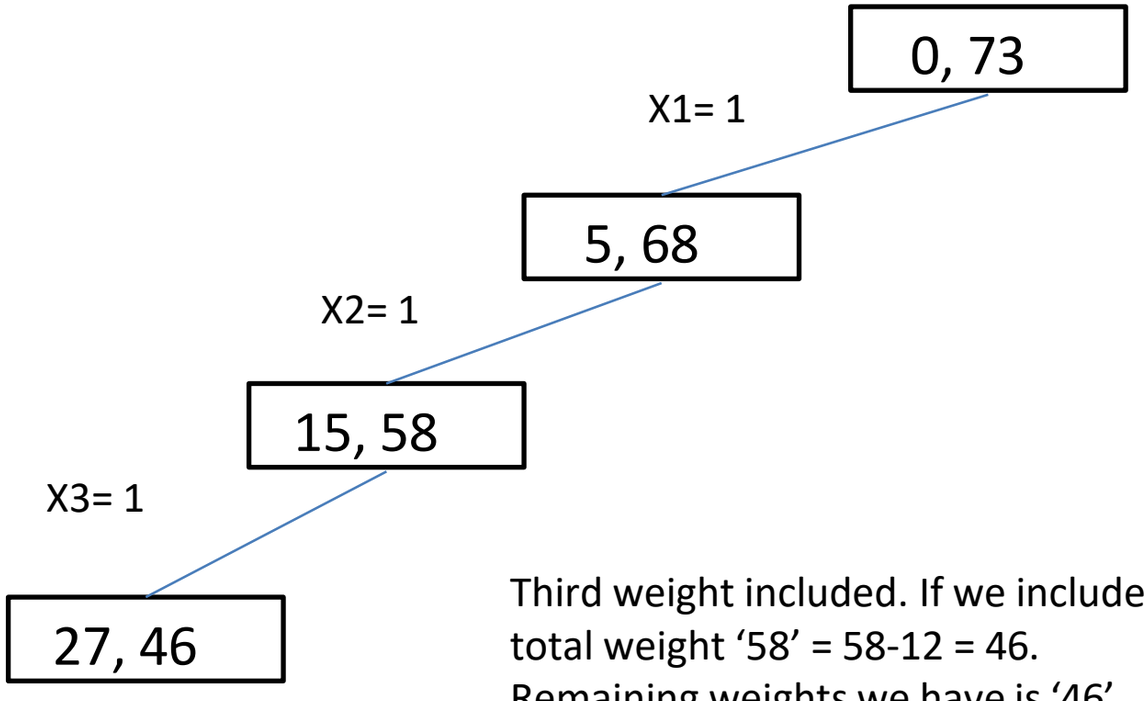
We have to consider whether we are including the weight or not so one by one we have to consider



W[1:6] = {5,10,12,13,15,18}

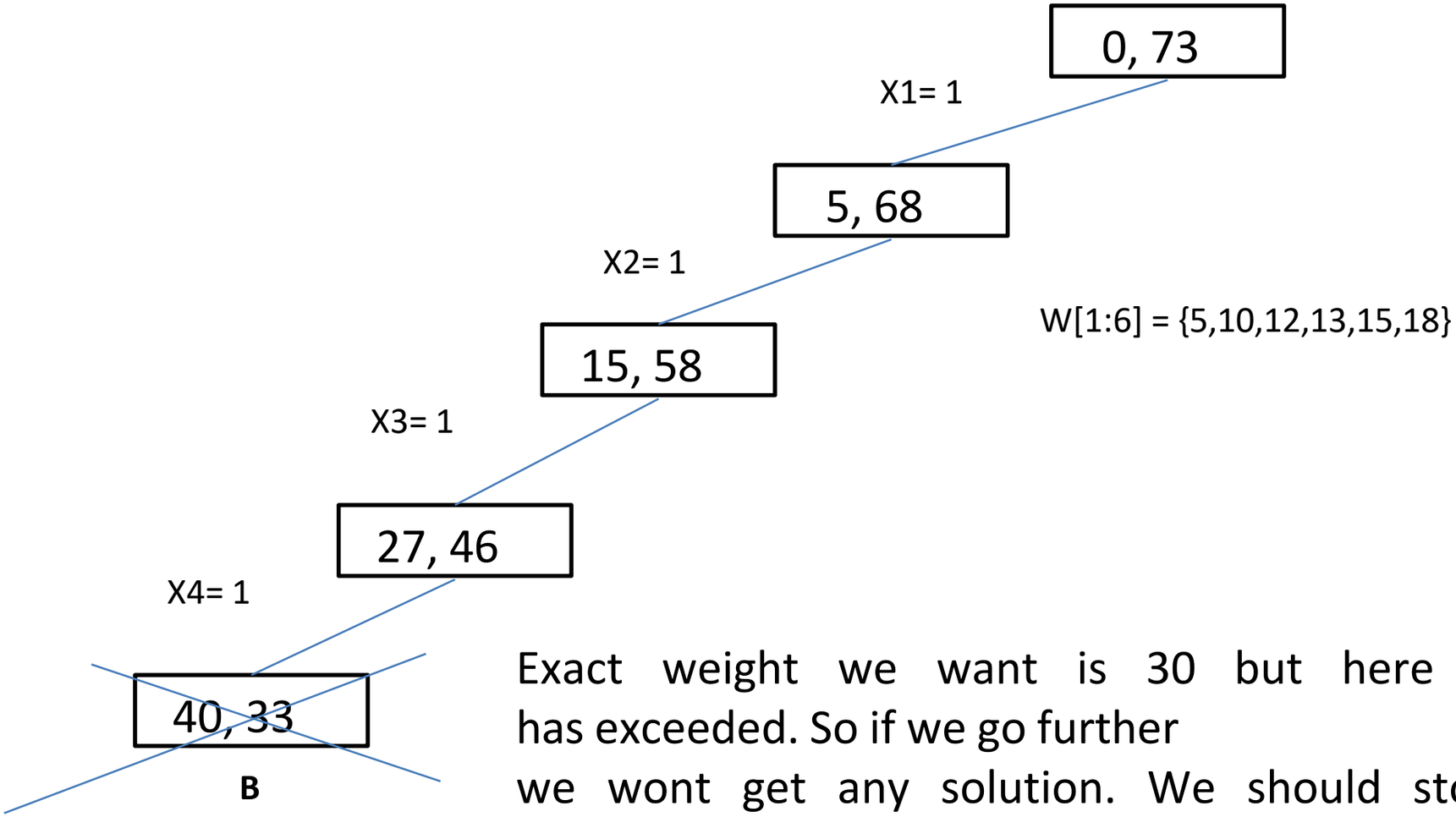
First weight included in the solution
Initially there was no weight now it become 5.
The total weight is 73 now 5 less from it so it will '68‘. So remaining weights we have is '68'

Second weight included. If we include it 5 +10 = 15. Subtract ‘10’ from total weight ‘68’ = 68-10 = 58.
Remaining weights we have is ‘58’



W[1:6] = {5,10,12,13,15,18}

Third weight included. If we include it 15 +12 = 27. Subtract ‘12’ from total weight ‘58’ = 58-12 = 46.
Remaining weights we have is ‘46’

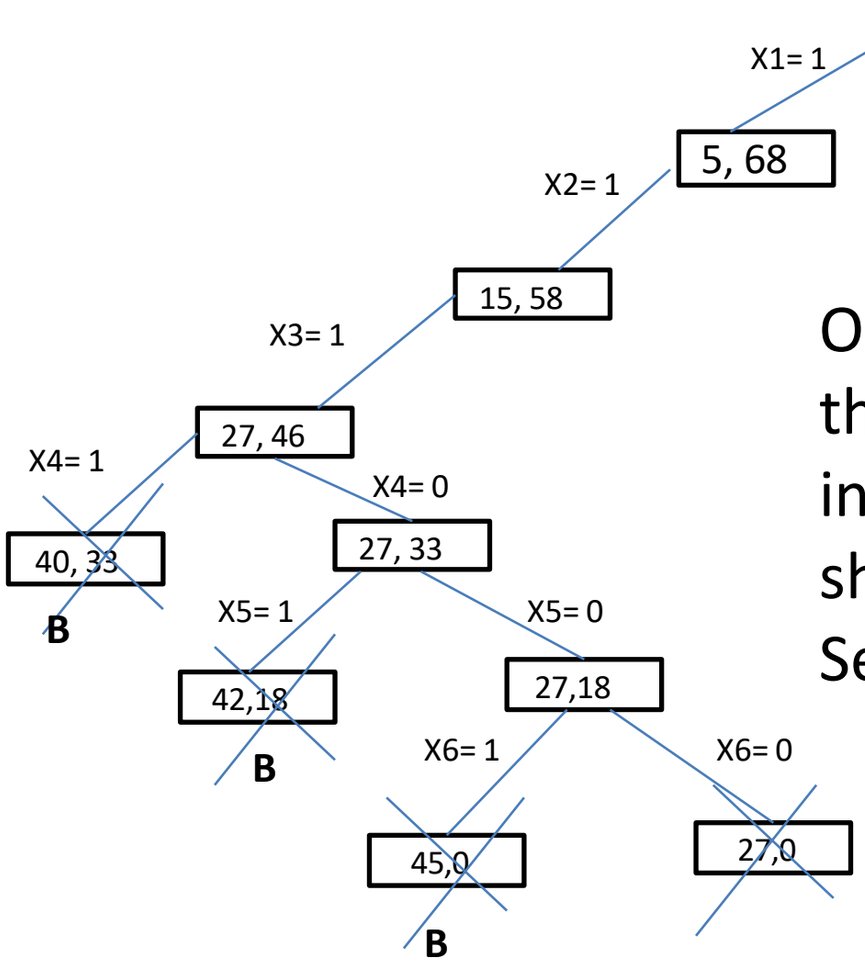


Exact weight we want is 30 but here it has exceeded. So if we go further
we wont get any solution. We should stop here and kill this node with some bounding function. Bounding function is a condition

- Sum of the weights included so far plus weight of next object should be less than or equal to 'm'
- If its greater then there is no need to continue

$$\sum_{i=1}^{k} w_i x_i + w_{k+1} <= m$$

We cant include 4 so try without including 4th object

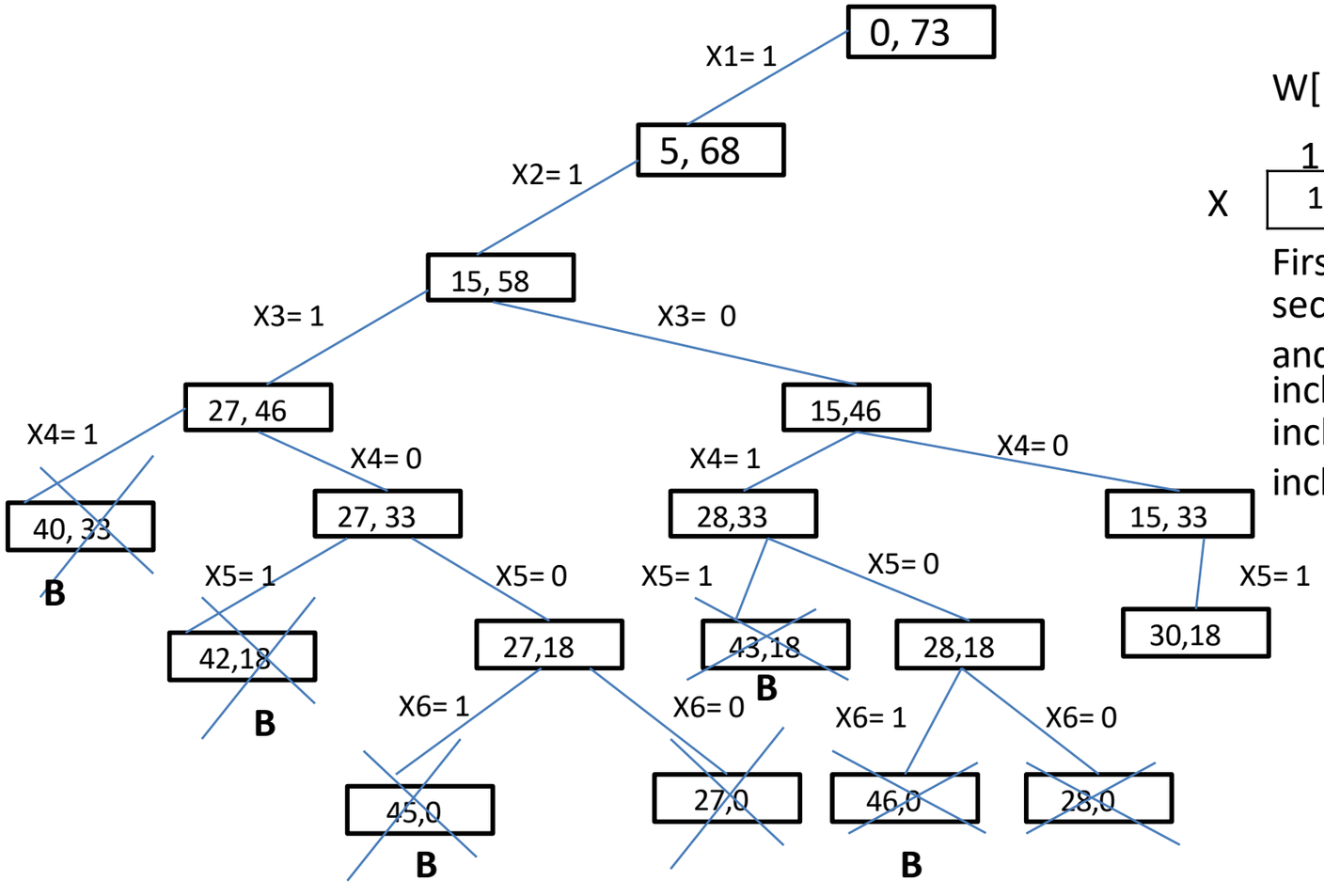


W[1:6] = {5,10,12,13,15,18}

One more bounding function is there So for what all you have included and remaining weights should be greater than m.

See 27 + 0 is not greater than 30

$$\sum_{i=1}^{k} w_i x_i + \sum_{k+1}^{n} w_i > m$$



W[1:6] = {5,10,12,13,15,18}

	1	2	3	4	5	6
X	1	1	0	0	1	0

First weight is included second is also included third and fourth weights is not included fifth weight is included sixth weight is not included

An iterative backtracking method is shown below

```
1   Algorithm IBacktrack(n)
2   // This schema describes the backtracking process.
3   // All solutions are generated in x[1 : n] and printed
4   // as soon as they are determined.
5   {
6       k := 1;
7       while (k ≠ 0) do
8       {
9           if (there remains an untried x[k] ∈ T(x[1], x[2], ...,
10              x[k − 1])  and B_k(x[1], ..., x[k]) is true) then
11          {
12              if (x[1], ..., x[k] is a path to an answer node)
13                  then write (x[1 : k]);
14              k := k + 1; // Consider the next set.
15          }
16          else k := k − 1; // Backtrack to the previous set.
17      }
18  }
```

In this problem, there is a given set with some integer elements. And another some value is also provided, we have to find a subset of the given set whose sum is the same as the given sum value.

Here backtracking approach is used for trying to select a valid subset when an item is not valid, we will backtrack to get the previous subset and add another element to get the solution.

Input and Output Input:
This algorithm takes a set of numbers, and a sum value.
The Set: {10, 7, 5, 18, 12, 20, 15}
The sum Value: 35
Output:
All possible subsets of the given set, where sum of each element for every subsets is same as the given sum value.
{10, 7, 18}
{10, 5, 20}
{5, 18, 12}
{20, 15}

Algorithm

subsetSum(set, subset, n, subSize, total, node, sum)

Input – The given set and subset, size of set and subset, a total of the subset, number of elements in the subset and the given sum.

Output – All possible subsets whose sum is the same as the given sum.

```
Begin
  if total = sum, then display
    the subset
    //go for finding next subset
    subsetSum(set, subset, , subSize-1, total-set[node], node+1, sum)
    return
  else
    for all element i in the set, do
      subset[subSize] := set[i]
      subSetSum(set, subset, n, subSize+1, total+set[i], i+1, sum) done
End
```

The Backtracking Algorithm for the Sum-of-Subsets Problem

Problem: Given n positive integers (weights) and a positive integer W, determine all combinations of the integers that sum to W.

Inputs: positive integer n, sorted (nondecreasing order) array of positive integers w indexed from 1 to n, and a positive integer W.

Outputs: all combinations of the integers that sum to W.

```
void sum_of_subsets (index i,
                     int weight, int total)
{
  if (promising(i))
     if (weight == W)
        cout << include[1] through include[i];
     else{
        include[i + 1] = "yes";                  // Include w[i + 1].
        sum_of_subsets(i + 1, weight + w[i +1], total - w[i + 1]);
        include[i + 1] = "no";                   // Do not include w[i + 1].
        sum_of_subsets(i + 1, weight, total - w[i + 1]);
     }
}

bool promising (index i);
{
  return (weight + total >= W) && (weight == W || weight + w[i + 1] <= W);
}
```

Branch and Bound

⇒ Its similar to ~~branch~~ backtracking in the sense that it also uses state space tree for solving the problem

⇒ This is one more Problem solving strategy

⇒ Solutions represented like a state space Tree.

⇒ Its useful for solving optimization Problem

⇒ It can be used for minimization Problem

⇒ Example Job sequencing with deadline

Jobs = { J_1, J_2, J_3, J_4 }

P = { 10, 5, 8, 3 }

d = { 1, 2, 1, 2 }

⇒ We can see how state space Tree is generated for the above eg:

⇒ There are 2 methods of generating a Tree that depends how you want the solution

⇒ Suppose I say that am doing { J_1, J_4 } this is my solution. Then the solution

Can be presented as subset of Jobs.

$S_1 = \{ J_1, J_4 \}$

$S_2 = \{ 1, 0, 0, 1 \}$ ⟹ Second Method

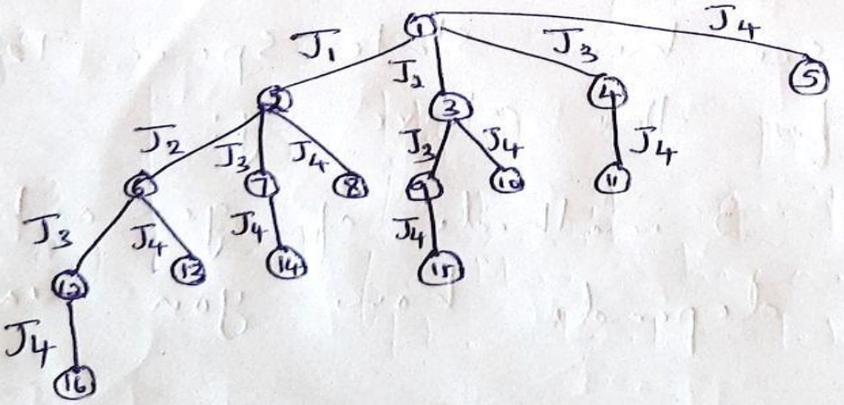
↳ Here 1st Job is done and 3rd Job not done & 4th Job done.

⟹ 'S_1' is Variable Size Solution

⟹ 'S_2' is Fixed Size Solution

As per the above set of Jobs example I got a set of 2 Jobs Sometimes it can be 3 Jobs. Sometimes it can be 4 Jobs. in the case of ~~Sa~~. solution 1 (S_1).

⟹ For Variable Size Solution $S_1 = \{ J_1, J_4 \}$ we can draw a State Space Tree

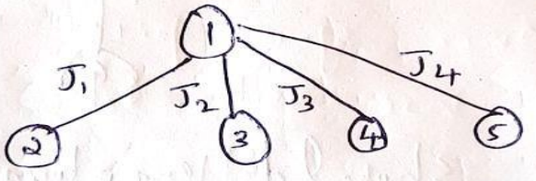


⇒ We are considering 1st Job then in backtracking we will consider the next level. but in branch & bound we will consider the second Job 'J_2' & 3rd Job 'J_3' & 4th Job 'J_4'.

⇒ So its like Breadth First Search. Here we don't go for DFS.

⇒ Backtracking was DFS & Branch and Bound is BFS.

⇒ Second method for the same Variable Size Solution



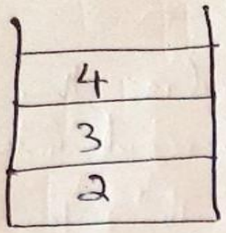


Stack

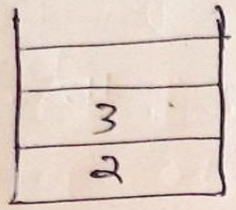
⇒ While generating the nodes we will put it in a stack.

⇒ Which node we have to expand next. It will be the 5th node because in the stack it will be the first Poped value.

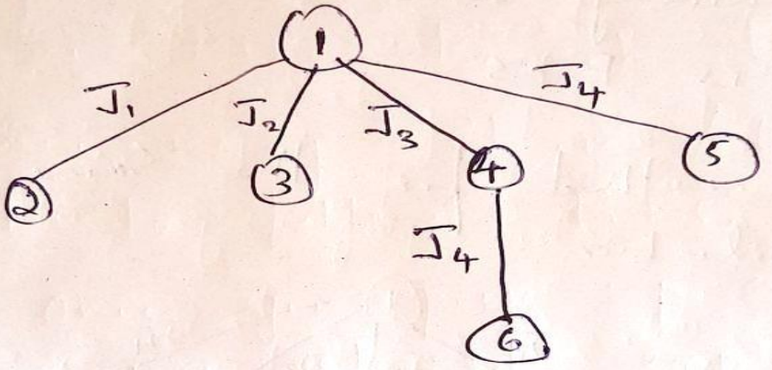
⇒ Already 5th node consist of J_4 which is the last Job so no expansion needed Pop out the next node from stack

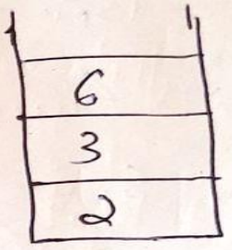


⇒ Here we Pop out 5

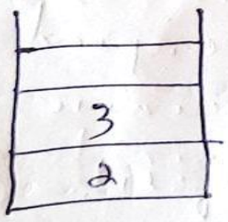


⇒ Here we pop out 4 & expand the Tree.

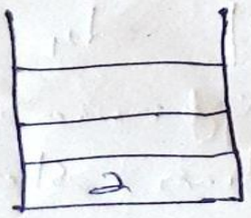




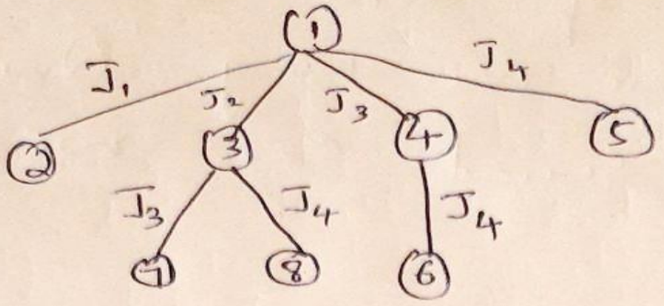
⇒ Next we should explore the 6th node.

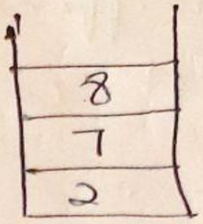


⇒ Here we pop out 6th node but it can't be expanded further because it contains the last dob 'J_4'.

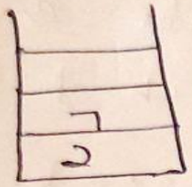
⇒ 

⇒ Pop out the value 3 for expansion

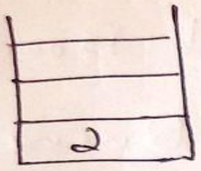




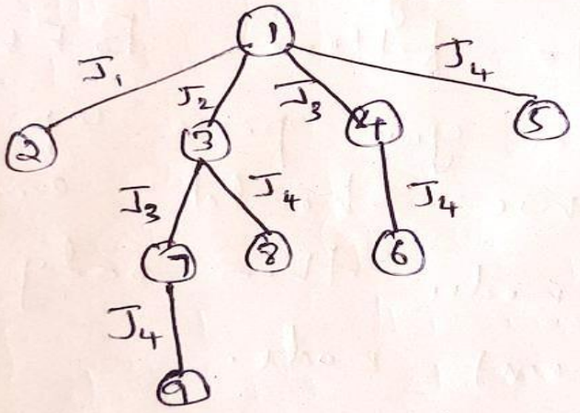
Puting 7 & 8 on stack

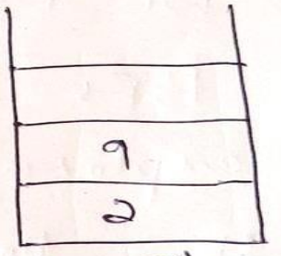
⟹ 

⟹ Pop out '8' node but it can't be expended further

⟹ 

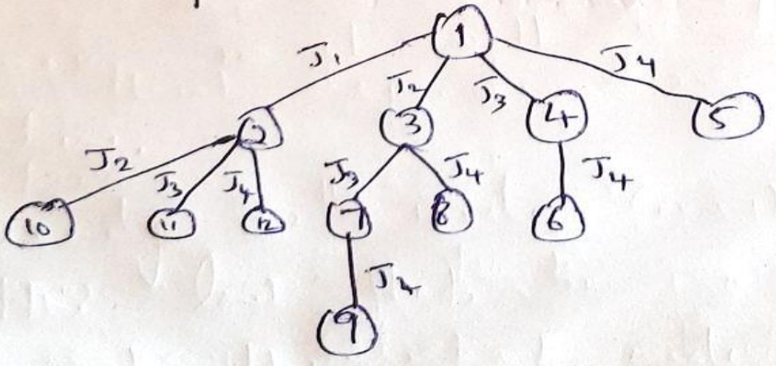
⟹ Pop out '7' node for expansion

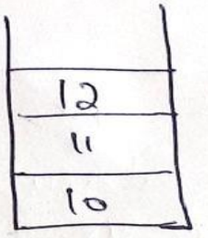




Push 9th node in to stack

⟹ While Pop out 9th node we can't expand so Pop out 2nd node from stack





⇒ Difference of both Solution are
1st case we are using 'Queue'
& in the second case we are
using Stack.

⇒ Queue for next node ~~node~~ exploration
is called as FIFO Branch & Bound

⇒ Stack for next node exploration is
called as LIFO Branch & Bound.

⇒ If you are using Stack / Queue too
we need to follow BFS.

⇒ BFS :- Once you pick up a node for
exploration we should explore all
the possible nodes then only you should
select the next node.

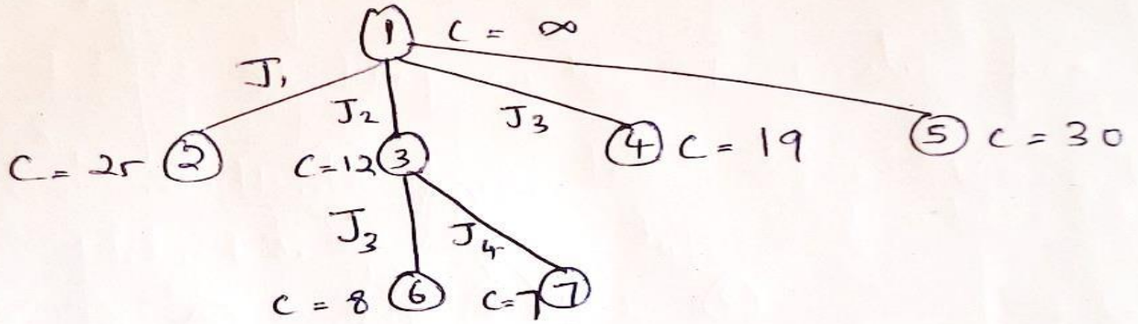
⇒ One more method is there ~~Least~~
"Least Cost Branch & Bound"

Branch & Bound

Least Cost – BB

⇒ For every node we will calculate the cost.

⇒ Depending on the problem we define a cost function & using that function we find out the cost of each node.



⇒ Next node exploration will start with a node contains least cost

So 3 methods are

- FIFO – BB
- LIFO – BB
- LC – BB

Lower Bounds: The Decision Tree method, Lower Bounds for Comparison based Sort and Searching

Lower Bound Theory

Lower Bound Theory Concept is based upon the calculation of minimum time that is required to execute an algorithm is known as a lower bound theory or Base Bound Theory.

Concept/Aim: The main aim is to calculate a minimum number of comparisons required to execute an algorithm.

The techniques which are used by lower Bound Theory are:

- Comparisons Trees.
- Oracle and adversary argument
- State Space Method

1. **Comparison trees:**

In a comparison sort, we use only comparisons between elements to gain order information about an input sequence (a1; a2.....an).

Given a_i,a_j from (a_1, a_2.....a_n)We Perform One of the Comparisons

- $a_i < a_j$ less than
- $a_i \leq a_j$ less than or equal to
- $a_i > a_j$ greater than
- $a_i \geq a_j$ greater than or equal to
- $a_i = a_j$ equal to

To determine their relative order, if we assume all elements are distinct, then we just need to consider $a_i \leq a_j$ '=' is excluded &, $\geq, \leq, >, <$ are equivalent.

Consider sorting three numbers a1, a2, and a3. There are 3! = 6 possible combinations:

(a1, a2, a3), (a1, a3, a2),

(a2, a1, a3), (a2, a3, a1)

(a3, a1, a2), (a3, a2, a1)

The Comparison based algorithm defines a decision tree.

Decision Tree: A decision tree is a full binary tree that shows the comparisons

between elements that are executed by an appropriate sorting algorithm operating on an input of a given size. Control, data movement, and all other conditions of the algorithm are ignored.

In a decision tree, there will be an array of length n.

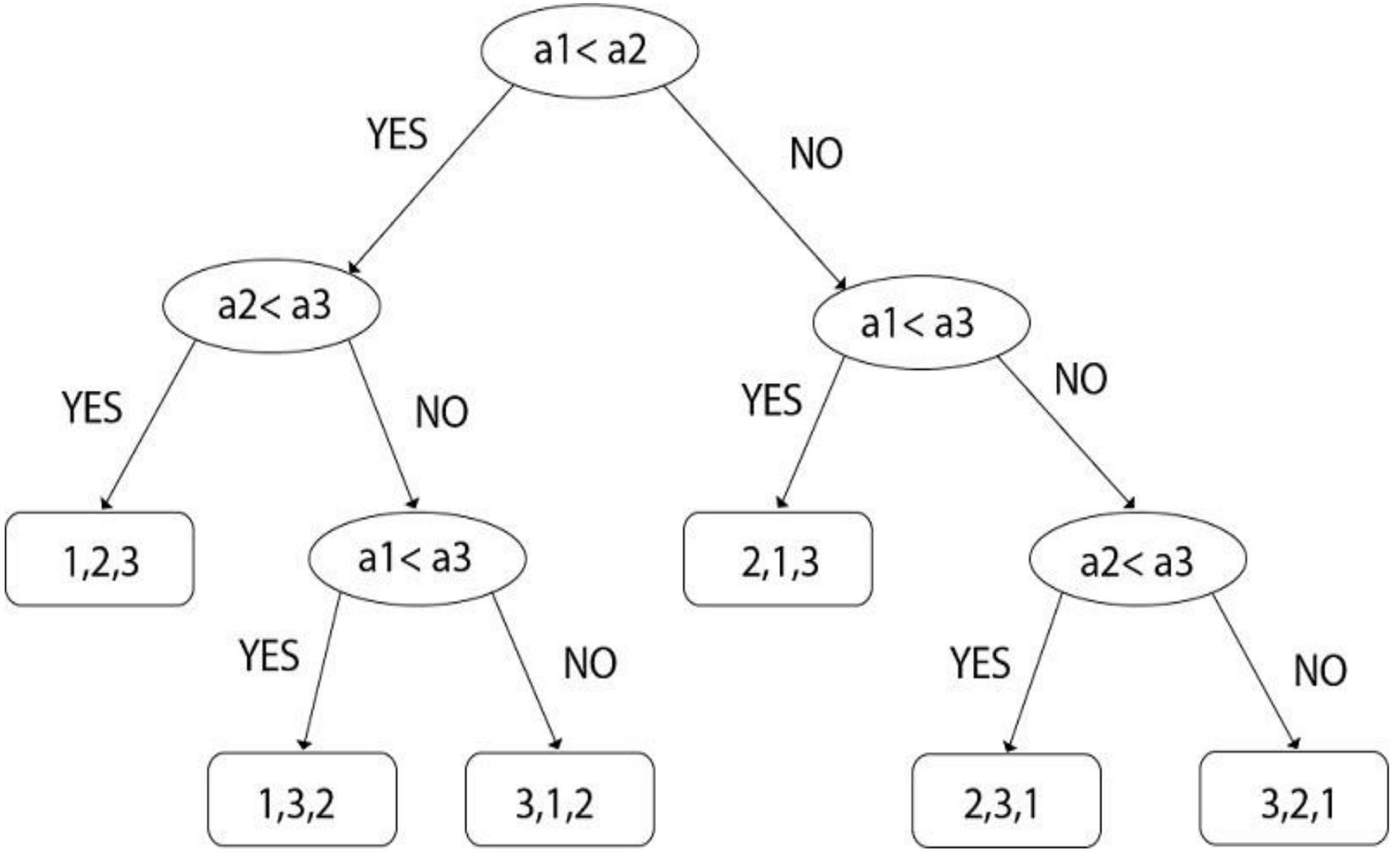
So, total leaves will be n! (I.e. total number of comparisons)

If tree height is h, then surely $n! \leq 2^n$

(tree will be binary)

Taking an Example of comparing a1, a2, and a3.

Left subtree will be true condition i.e. $a_i \leq a_j$ Right subtree will be false condition i.e. $a_i > a_j$



Comparison tree for Binary Search:

Example: Suppose we have a list of items according to the following Position:
1,2,3,4,5,6,7,8,9,10,11,12,13,14

$$\text{Mid} = \left[\left(\frac{1+14}{2}\right)\right] = \frac{15}{2} = 7.5 = \mathbf{7}$$

Note: Choose the greatest integer

1, 2, 3, 4, 5, 6	8, 9, 10, 11, 12, 13, 14
$\text{Mid}=\left(\frac{1+6}{2}\right)=\mathbf{3}$	$\text{Mid}=\left(\frac{8+14}{2}\right)=\mathbf{11}$

1, 2	4, 5, 6
$\text{Mid}=\left(\frac{1+2}{2}\right)=\mathbf{1}$	$\text{Mid}=\left(\frac{4+6}{2}\right)=\mathbf{5}$

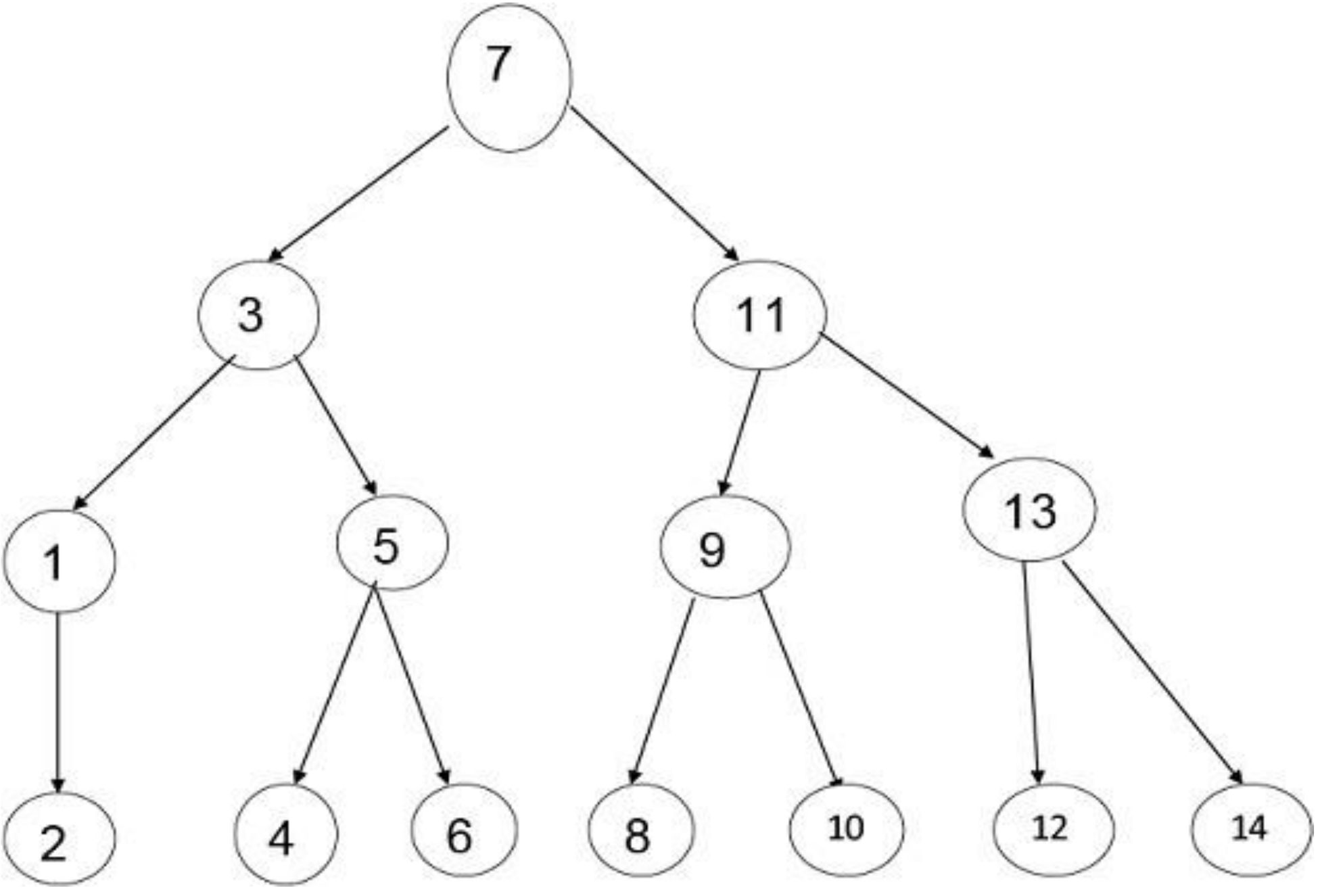
8, 9, 10	12, 13, 14
$\text{Mid}=\left(\frac{8+10}{2}\right)=\mathbf{9}$	$\text{Mid}=\left(\frac{12+14}{2}\right)=\mathbf{13}$

And the last midpoint is:

2, 4, 6, 8, 10, 12, 14

Thus, we will consider all the midpoints and we will make a tree of it by having stepwise midpoints.

According to Mid-Point, the tree will be:



Step 1: Maximum number of nodes up to k level of the internal node is 2^k-1

For Example

2^k-1

$2^3-1= 8-1=7$Where k = level=3

Step 2: Maximum number of internal nodes in the comparisons tree is n!

Step 3: From Condition1 & Condition 2 we get

$N! \leq 2^k-1$

$14 < 15$ Where N = Nodes

Step 4: Now, $n+1 \leq 2^k$

Here, Internal Nodes will always be less than 2^k in the Binary Search.

Step5:

$n+1 <= 2^k$

$\text{Log}\ (n+1) = k \log 2$

$$k >= \frac{\log(n+1)}{\log 2}$$

$k >= \log_2(n+1)$

Step6:

$T(n) = k$

Step7:

$T(n) >= \log_2(n+1)$

Here, the minimum number of Comparisons to perform a task of the search of n terms using Binary Search

Lower bound of Comparison based Sorting

The problem of sorting can be viewed as following.

Input: A sequence of n numbers $<a_1, a_2, \ldots, a_n>$.

Output: A permutation (reordering) $<a'_1, a'_2, \ldots, a'_n>$ of the input sequence such that $a'_1 <= a'_2 \ldots <= a'_n$.

A sorting algorithm is comparison based if it uses comparison operators to find the order between two numbers. Comparison sorts can be viewed abstractly in terms of decision trees.

A decision tree is a full binary

tree that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size. The execution of the sorting algorithm corresponds to tracing a path from the root of the decision tree to a leaf. At each internal node, a comparison $a_i <= a_j$ is made. The left subtree then dictates subsequent

comparisons for $a_i <= a_j$, and the right subtree dictates subsequent comparisons for $a_i > a_j$. When we come to a leaf, the sorting algorithm has established the ordering. So we can say following about the decision tree.

1)Each of the $n!$ permutations on n elements must appear as one of the leaves of the decision tree for the sorting algorithm to sort properly.

2)Let x be the maximum number of comparisons in a sorting algorithm. The maximum height of the decision tree would be x. A tree with maximum height x has at most 2^x leaves.

After combining the above two facts, we get following relation.

$n! <= 2^x$

Taking Log on both sides.

$\log_2(n!) <= x$

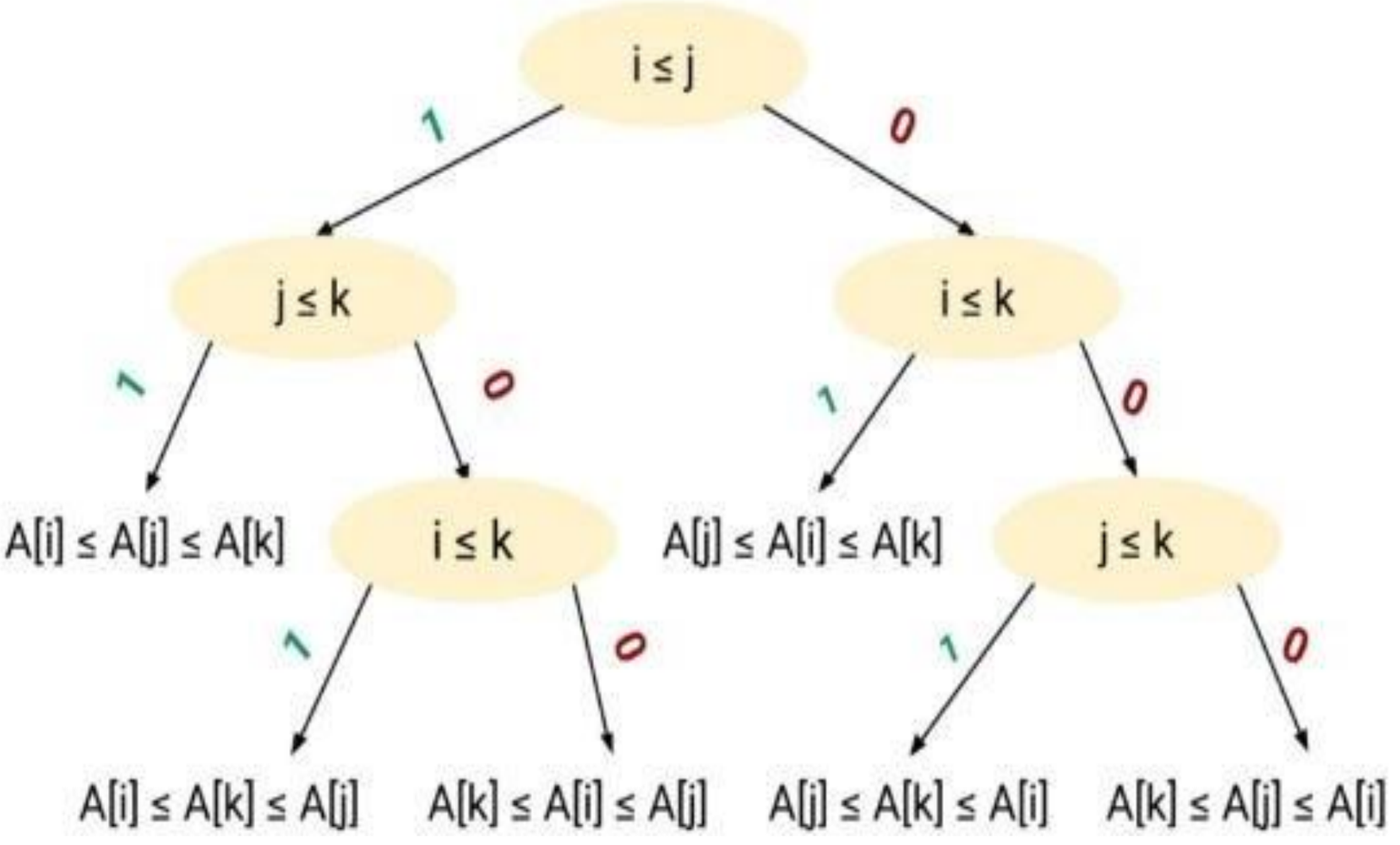
Since $\log_2(n!) = \Theta(n\text{Log}n)$, we can say

$x = \Omega(n\text{Log}_2 n)$

Lower Bound on Comparison-based Algorithms

- Theorem:
  - Suppose that an algorithm uses ONLY comparison of keys to solve a problem.
  - If there are "n" possible outcomes, then the algorithm must take AT LEAST $\log_2 N$ comparison of keys in the worst case to solve the problem.

Why lower bound of **comparison sort** is O(nlogn)?

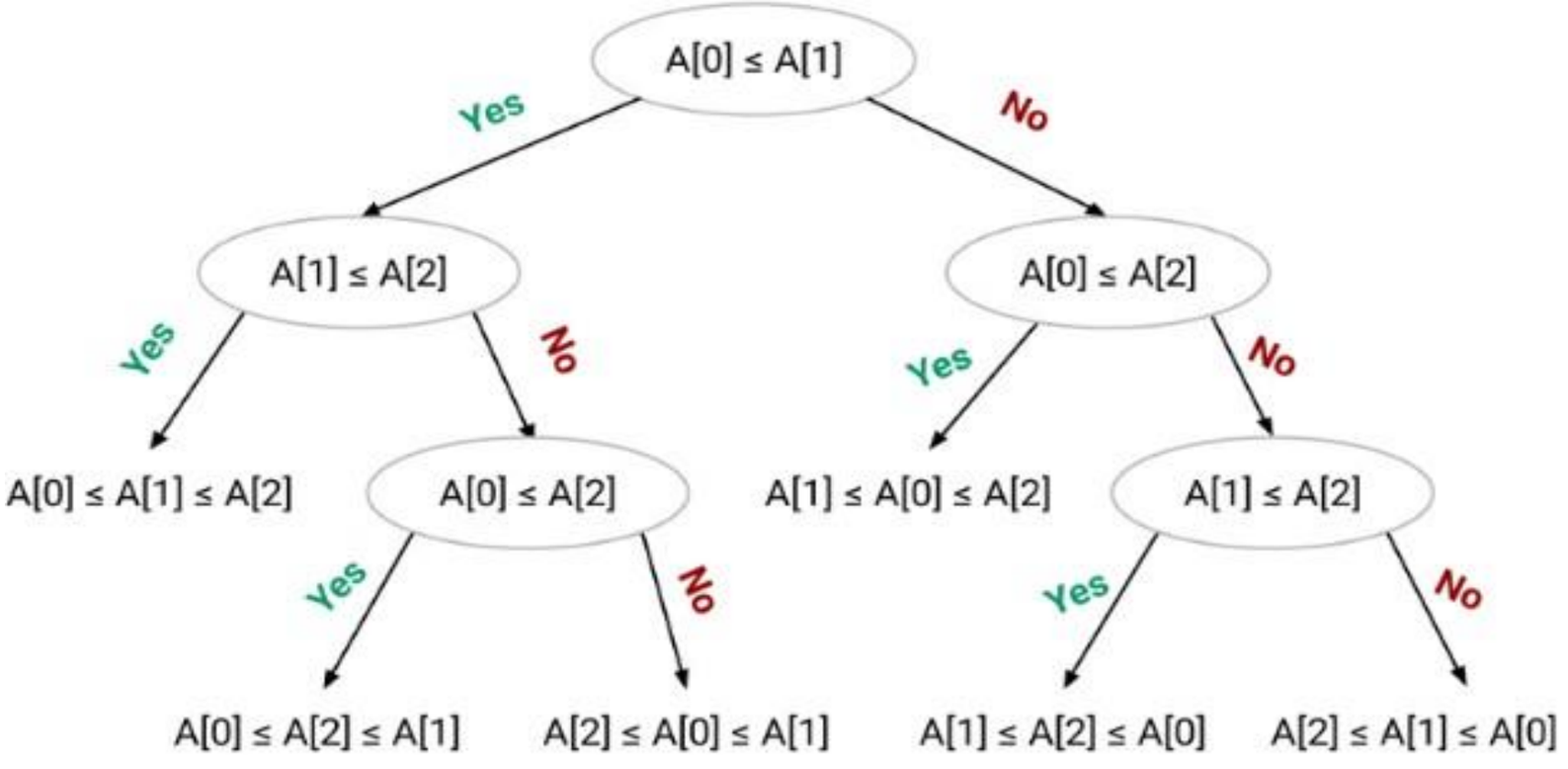


We have already discussed several sorting algorithms that can sort n numbers in both $O(n^2)$ and O(nlogn) time complexity in the worst case. These algorithms determine the sorted order based only on the comparisons between the input elements. We call such sorting algorithms **comparison sort**, where we use only comparisons between elements to gain order information about an input sequence A[0], A[1], . . . , A[n-1]. In other words, given two elements A[i] and A[j], we perform one of the tests A[i] <= A[j], A[i] >= A[j], A[i] = A[j] to determine their relative order. Here we assume without loss of generality that all of the input elements are distinct.

Decision tree model of comparison sorting

We can visualize comparison sorts in terms of decision trees. A decision tree is a full binary tree structure that represents the comparisons performed by a sorting algorithm when it operates on an input of a given size. Here we only consider comparison operation and ignore other operations like control, data movement, etc.

Here is an example of a decision tree model for insertion sort operating on three elements. There are 3! = 6 possible permutations of the input elements, so the decision tree must have at least 6 leaves. Think!



Each internal node in the decision tree is represented by a comparison between a pair of elements (A[i], A[j]) for some i and j in the range $0 \leq i, j \leq n-1$. if $A[i] \leq A[j]$ is true, we follow the left path, otherwise, we follow the right path.

Each leaf node represents a permutation of the given input. In other words, each of the n! permutations on n elements must appear as one of the leaves of the decision tree for the sorting algorithm to sort properly.

The execution of the sorting algorithm corresponds to tracing a path from the root of the decision tree to a leaf. At each internal node, a comparison $A[i] \leq A[j]$ is made. The left subtree then dictates subsequent comparisons for $A[i] \leq A[j]$ and the right subtree dictates subsequent comparisons for $A[i] > A[j]$. When we come to a leaf, the sorting algorithm has established the sorted ordering of the input elements.

The height of the decision tree (length of the longest path from the root to any of its leaves) represents the worst-case number of comparisons the sorting algorithm performs. A lower bound on the heights of decision trees is therefore a lower bound on the running time of any comparison sort algorithm.